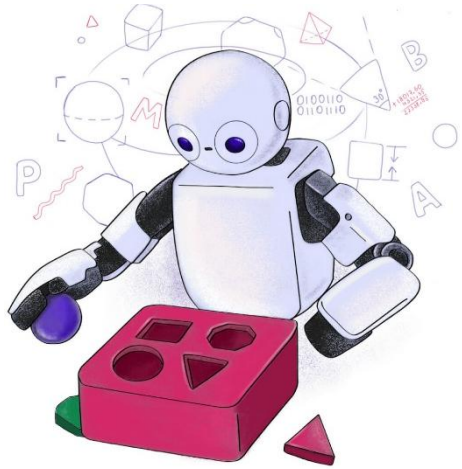
TP557 - Tópicos avançados em IoT e Machine Learning: *TensorFlow Lite*

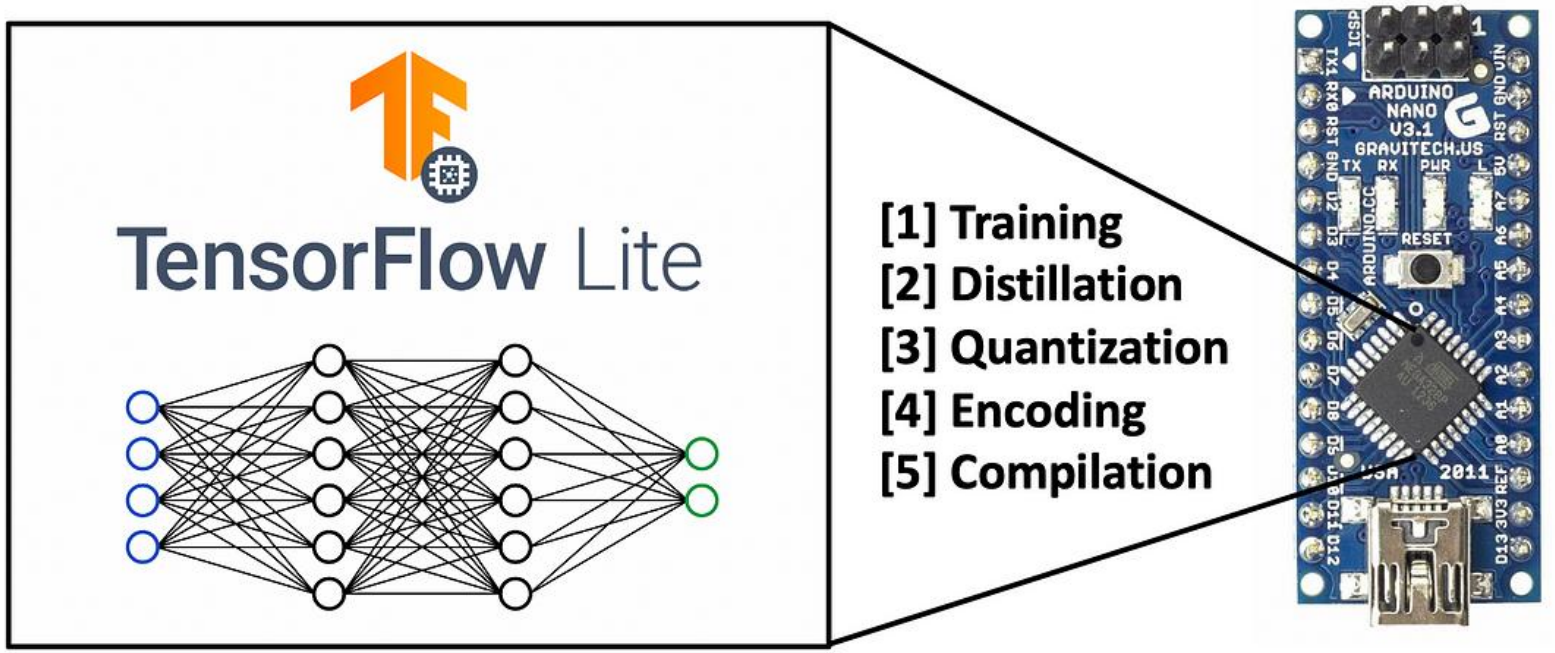


Inatel

Créditos: Prof. Marcelo Rovai UNIFEI

Samuel Baraldi Mafra
samuelbmafra@inatel.br

TinyML



TinyML não é apenas colocar uma rede neural em um microcontrolador. É todo um processo de otimização, conversão e implantação para que modelos de Machine Learning consigam funcionar em ambientes extremamente restritos.

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Processamento na borda com dispositivos embarcados

- Características dos dispositivos IoT:
  - Baixo consumo de energia;
  - Baixo poder computacional;
  - Baixo custo.

Essas limitações criam um desafio:

Como executar inteligência artificial em dispositivos tão restritos?

→ É exatamente nesse contexto que surge o TinyML, que busca adaptar modelos de Machine Learning para funcionar em microcontroladores e outros sistemas embarcados de baixo consumo.

Processamento na borda com dispositivos embarcados

- A energia de processamento depende basicamente do número de ciclos de *clock* executados pelo microcontrolador.

- "Os programadores ***não poderão ignorar*** o custo de energia dos programas que escrevem... Você precisa de ferramentas que forneçam *feedback* e digam o quão boas são suas decisões. Atualmente, as ferramentas não fornecem esse tipo de feedback." (Steve Furber, ARM, 2010)

→ Em IoT e TinyML, eficiência energética é parte do projeto do software.

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Processamento tradicional

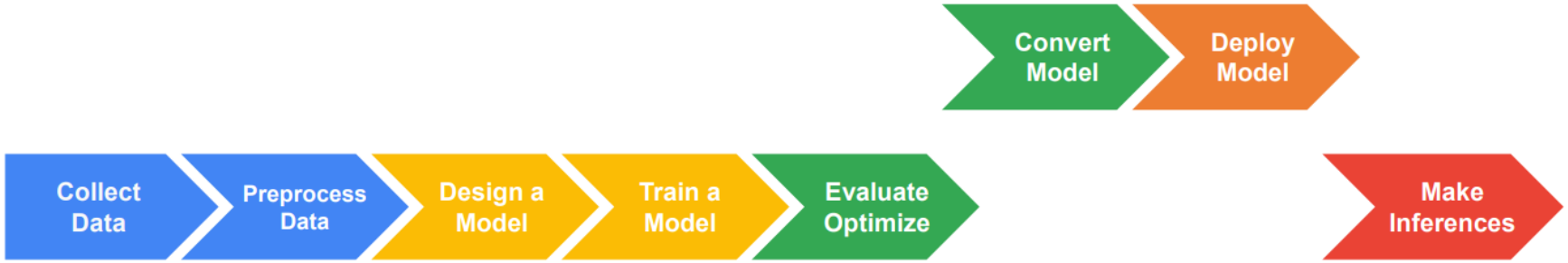


→ No processamento tradicional, o modelo é desenvolvido, treinado e avaliado em ambientes com maior poder computacional. A inferência também pode ocorrer em servidores ou na nuvem.

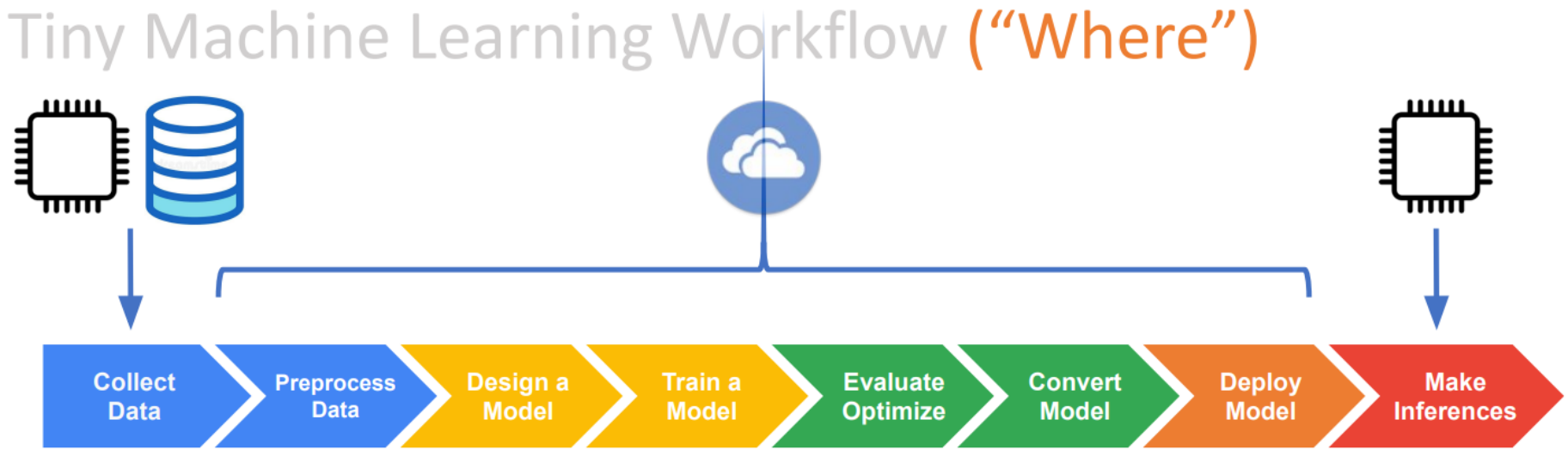
Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Processamento na borda com dispositivos embarcados

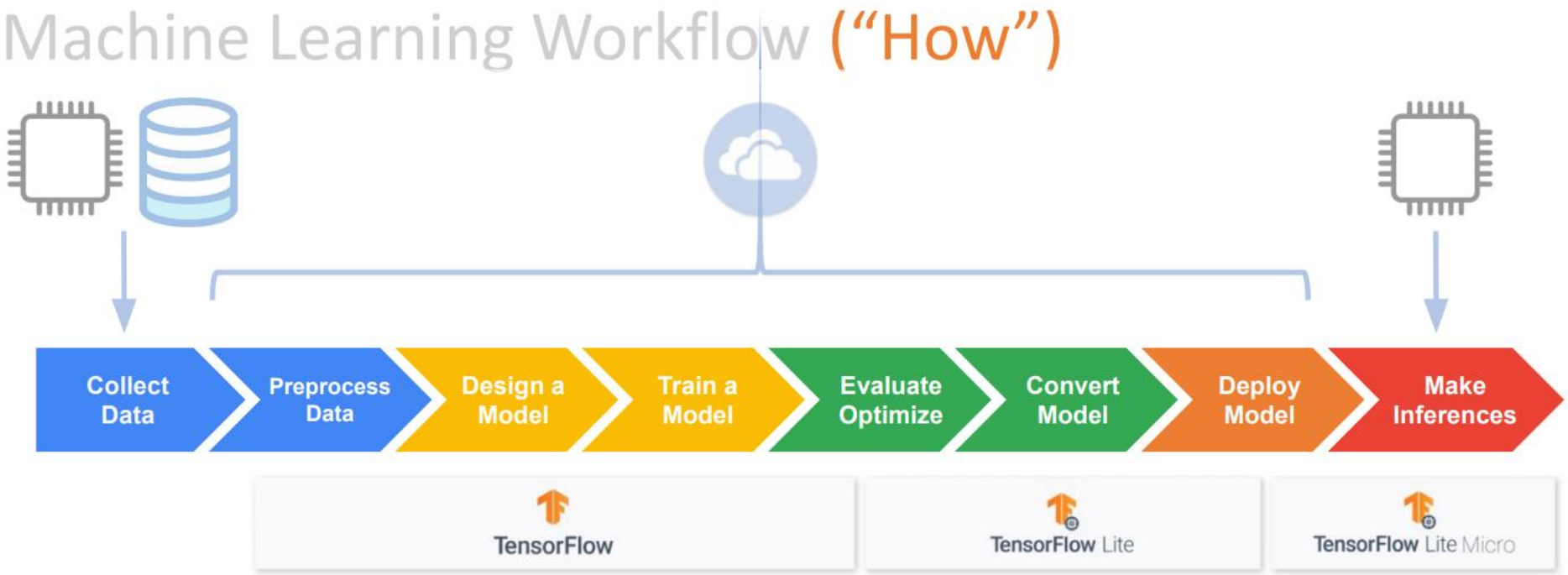
Tiny Machine Learning Workflow ("What")



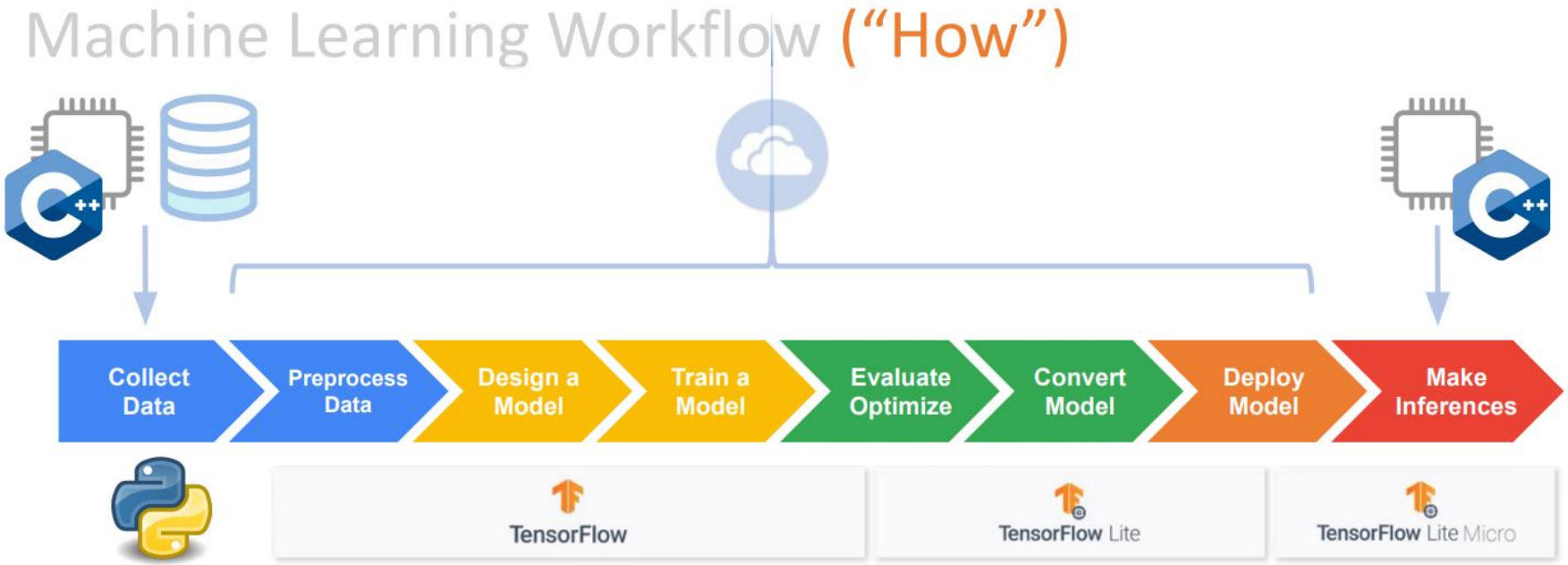
Processamento na borda com dispositivos embarcados



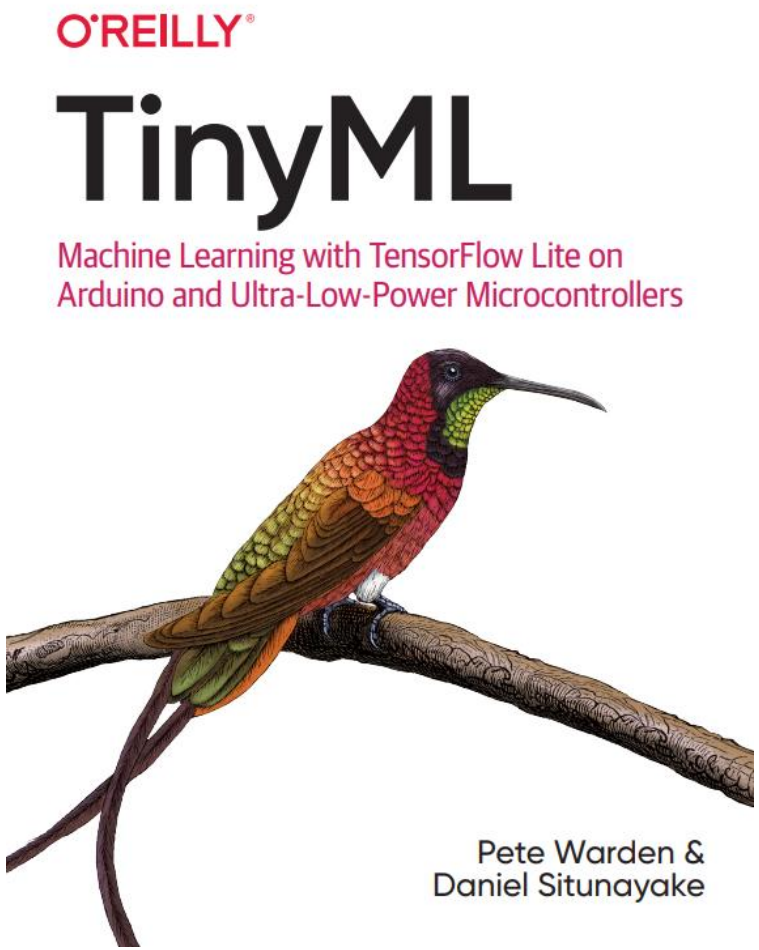
Processamento na borda com dispositivos embarcados



Processamento na borda com dispositivos embarcados



Referência



https://tinyurl.com/edgemlbooks

Tinyml: Machine Learning with Tensorflow Lite on Arduino and Ultra-Low-Power Microcontrollers

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Conteúdo

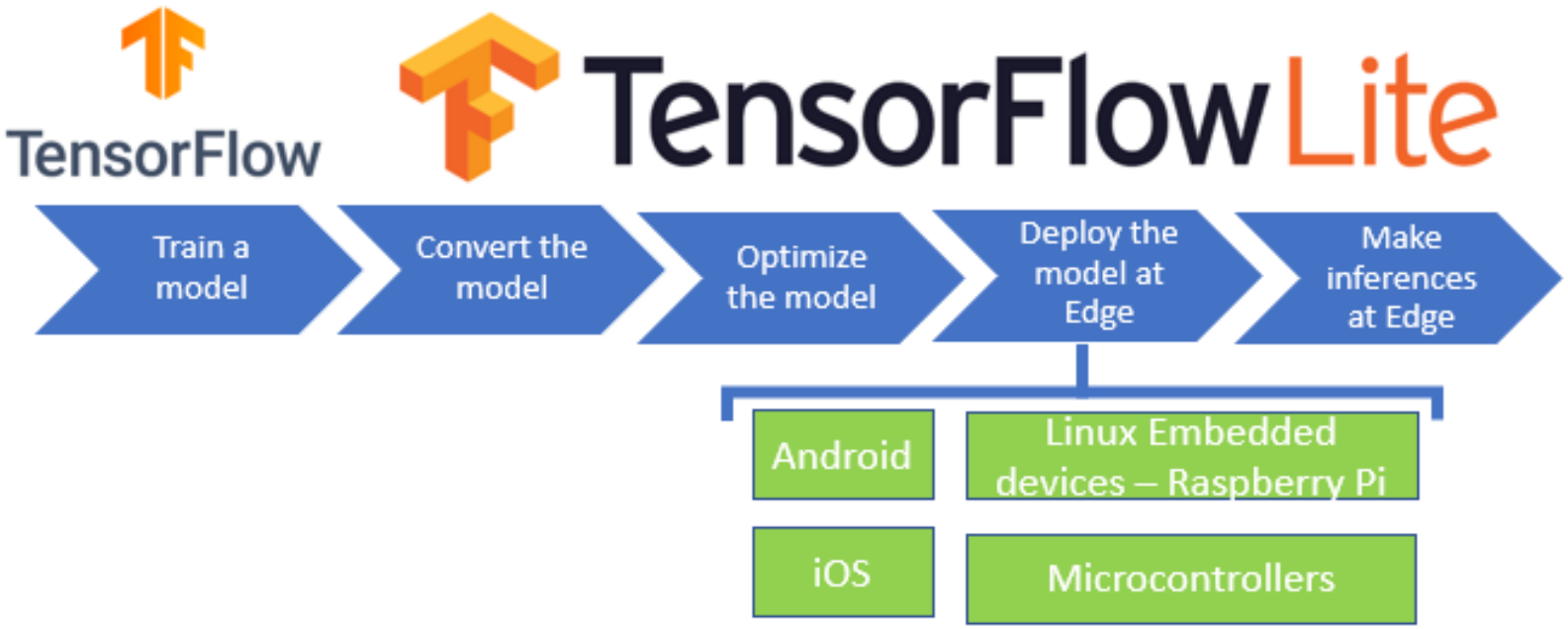
→ Introdução

- O que é o TensorFlow Lite?
- Por que usar o TensorFlow Lite?
- Benefícios de usar o TensorFlow Lite
- O que o TensorFlow Lite pode fazer?
- Visão geral do fluxo de trabalho do TensorFlow Lite
- Convertendo um modelo do TensorFlow em TensorFlow Lite

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

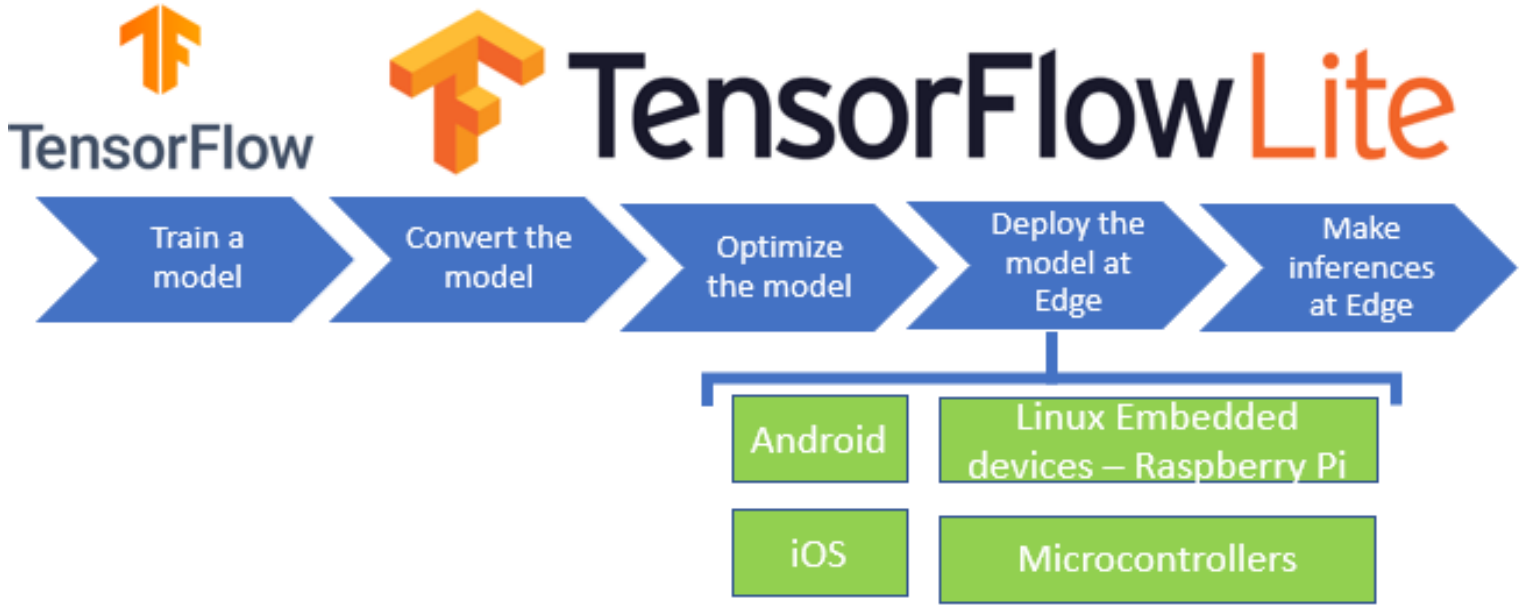
- O TensorFlow Lite é um ***conjunto de ferramentas*** que ajuda os desenvolvedores a ***executar modelos de machine learning em dispositivos embarcados*** (móveis e IoT).



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

- O TensorFlow é uma biblioteca de aprendizado de máquina que pode ser usada para ***treinar, implantar e executar modelos de aprendizado de máquina em uma variedade de plataformas sem restrições computacionais***.
- O TensorFlow Lite é uma versão reduzida do TensorFlow que é otimizada para desempenho e eficiência em dispositivos embarcados com ***recursos moderados***.
  - Tensorflow Lite Micro: o foco são microcontroladores e dispositivos ***extremamente limitados em recursos*** (sem sistema operacional, com pouca memória e poder de processamento).



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

Por que usar o TensorFlow Lite?

- É otimizado para eficiência computacional e de memória.
- Inclui APIs para Android, iOS e Python.
- Pode usar *delegates* para acelerar operações em hardware específico (ex: GPU, DSP, Coral Edge TPU).
- Compatibilidade com diversas linguagens, incluindo Java, Swift, Objective-C, C++ e Python

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

Por que usar o TensorFlow Lite?

- Auxilia na criação de aplicações edgeML, que possuem as seguintes características:
  - Baixa latência (não há ida e volta para um servidor),
  - Maior privacidade e segurança (nenhum dado pessoal sai do dispositivo),
  - Pouca ou nenhuma necessidade de conectividade com a Internet,
  - Modelos com menor tamanho,
  - Menor consumo de energia (inferência eficiente e falta de conexões de rede)

TensorFlow Lite

	TensorFlow	TensorFlow Lite	
Topology	Variable	Fixed	Não suporta treinamento
Weights	Variable	Fixed	Pesos **fixos**
Binary Size	Unimportant	High Priority	Biblioteca minimalista
Distributed Compute	Needed	Not Needed	Inferência local em um único dispositivo. Não faz treino/inferência em vários dispositivos
Developer Background	ML Researcher	Application Developer	Não apresenta flexibilidade para treinar e experimentar

TensorFlow Lite

	TensorFlow	TensorFlow Lite	TensorFlow Lite Micro
Training	Yes	No	No
Inference	Yes *(but inefficient on edge)*	Yes *(and efficient)*	Yes *(and even **more** efficient)*
How Many Ops	~1400	~130	~50
Native Quantization Tooling + Support	No	Yes	Yes

Set of operations: https://www.tensorflow.org/mlir/tfl_ops

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

	TensorFlow		TensorFlow Lite		TensorFlow Lite Micro	
Needs an OS	Yes		Yes	Android, iOS ou Linux embarcado	No	Roda em bare metal
Memory Mapping of Models	No	(mapeamento dinâmico de memória: memoria virtual)	Yes	Mapeamento estático de memória. Sabemos onde está na memória.	Yes	Otimização extrema para operar com pouca RAM, evitando alocações dinâmicas
Delegation to accelerators	Yes	(GPUs e TPUs)	Yes	(GPUs móveis, NPUs, DSPs)	No	Roda apenas no MCU

TensorFlow Lite

	TensorFlow	TensorFlow Lite	TensorFlow Lite Micro
Base Binary Size	3MB+	100KB	~10 KB
Base Memory Footprint	~5MB	300KB	20KB
Optimized Architectures	X86, TPUs, GPUs	Arm Cortex A, x86	Arm Cortex M, DSPs, MCUs

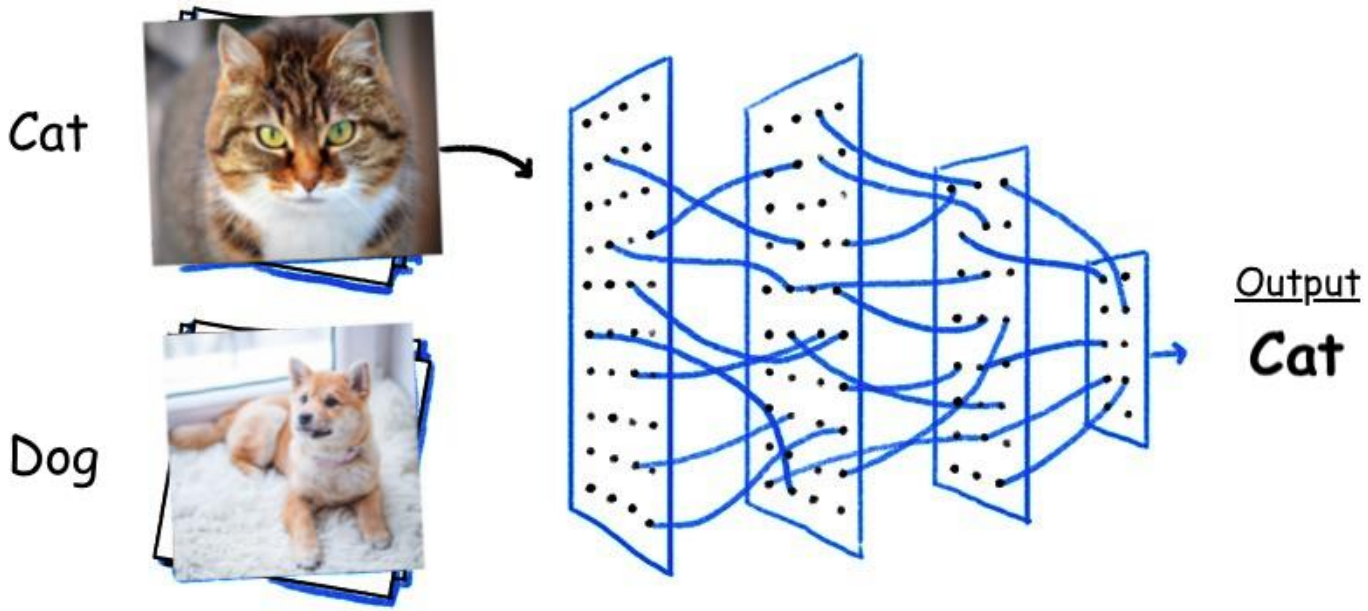
Base binary size: tamanho em disco ou memória flash que a biblioteca ocupa.
Memory footprint: quantidade total de memória RAM usada durante a execução.

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

O TensorFlow Lite pode ser usado para criar aplicações edgeML destinadas a uma ampla variedade de tarefas, incluindo:

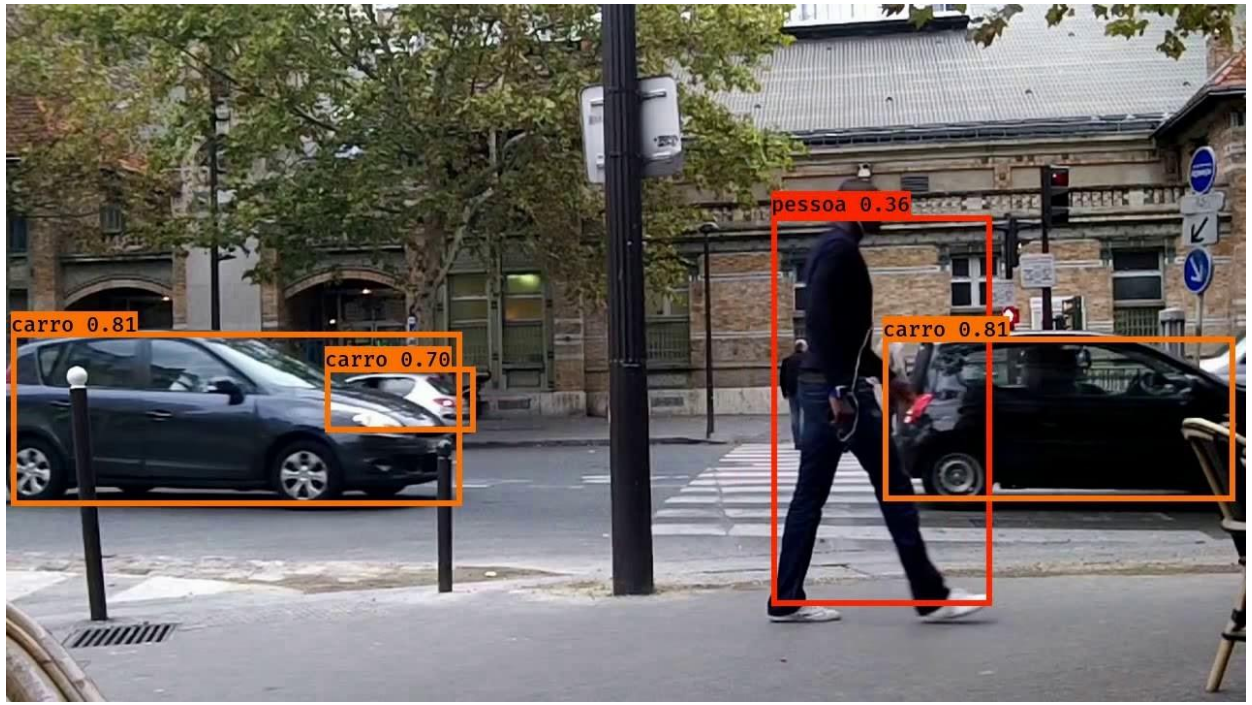
- Classificação de imagem: O TensorFlow Lite pode ser usado para criar modelos que classifiquem imagens.



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

Detecção, rastreamento e segmentação de objetos: O TensorFlow Lite pode ser usado para criar modelos que identifiquem, segmentem e rastreiem objetos em tempo real, como carros, pessoas e animais.



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

Reconhecimento de voz: O TensorFlow Lite pode ser usado para criar modelos de reconhecimento de fala, como comandos de voz e perguntas.



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

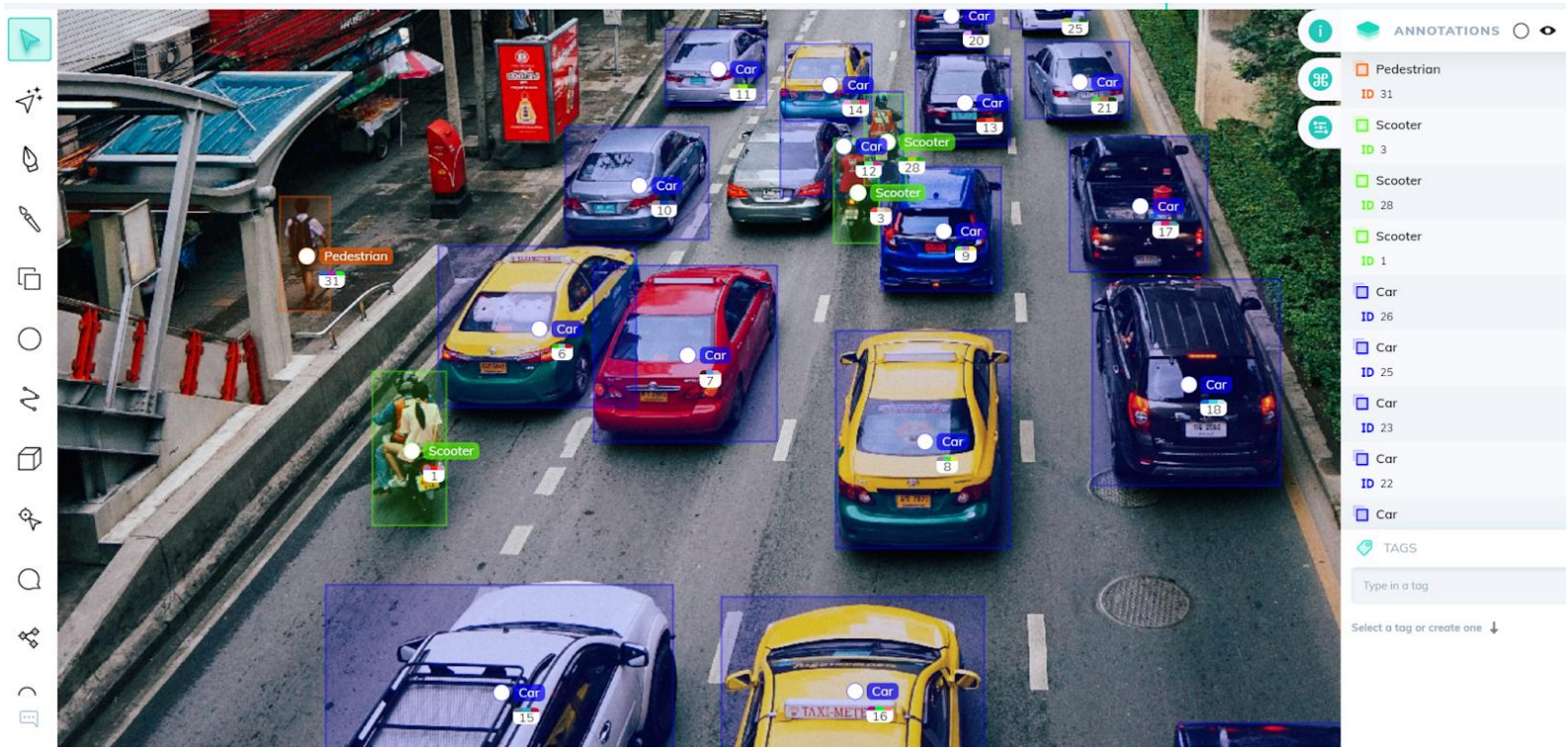
TensorFlow Lite

Processamento de linguagem natural: O TensorFlow Lite pode ser usado para criar modelos que entendam texto, como perguntas e solicitações.



TensorFlow Lite

Análise de vídeo: O TensorFlow Lite pode ser usado para criar modelos que analisem vídeos para detectar objetos, pessoas e comportamentos.



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

- Um modelo do TensorFlow Lite é representado em um ***formato aberto e eficiente*** conhecido como ***FlatBuffers***, identificado pela extensão de arquivo .tflite.
  - FlatBuffers: biblioteca e formato desenvolvidos pelo Google.
- É uma biblioteca de ***serialização*** multiplataforma eficiente para C++, C, Python, etc.
  - **Biblioteca que converte objetos em um formato que possa ser armazenado ou transmitido**.
- Flatbuffers é especialmente útil em cenários em que a ***velocidade de transferência e o tamanho compacto dos dados são essenciais***, como jogos, aplicativos móveis e sistemas de comunicação de rede.

TensorFlow Lite: Geração de modelos

É possível gerar um modelo do TensorFlow Lite das seguintes maneiras:

Usando um ***modelo existente*** do TensorFlow Lite: consulte os exemplos de TensorFlow Lite para escolher um modelo existente. Os modelos podem ou não conter metadados*.



*Os metadados do modelo contêm informações gerais do modelo (autor, versão, propósito, licença), dimensões esperadas das entradas/saídas, tipo dos dados (float, int), descrição das saídas, mapeamento dos rótulos em classes, taxa de amostragem de áudio, etc. o que torna o modelo autodescritivo e intercambiável.

https://github.com/tensorflow/examples/tree/master/lite

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

É possível gerar um modelo do TensorFlow Lite das seguintes maneiras:

Criando um modelo do TensorFlow Lite: use o ***TensorFlow Lite Model Maker*** para criar um modelo com seu próprio conjunto de dados personalizado.

Por padrão, todos os modelos contêm metadados*.

Usa ***transfer learning para reduzir a quantidade de dados*** de treinamento necessários e ***reduzir o tempo de treinamento***.

https://www.tensorflow.org/lite/models/modify/model_maker?hl=pt-br

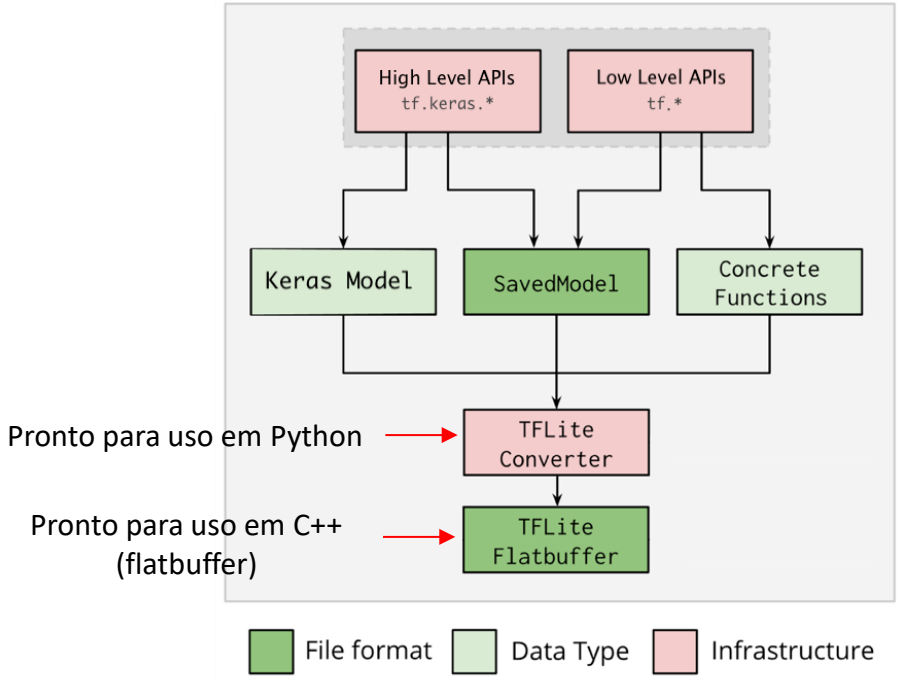
*Os metadados do modelo contêm informações gerais do modelo (autor, versão, propósito, licença), dimensões esperadas das entradas/saídas, tipo dos dados (float, int), descrição das saídas, mapeamento dos rótulos em classes, taxa de amostragem de áudio, etc. o que torna o modelo autodescritivo e intercambiável.

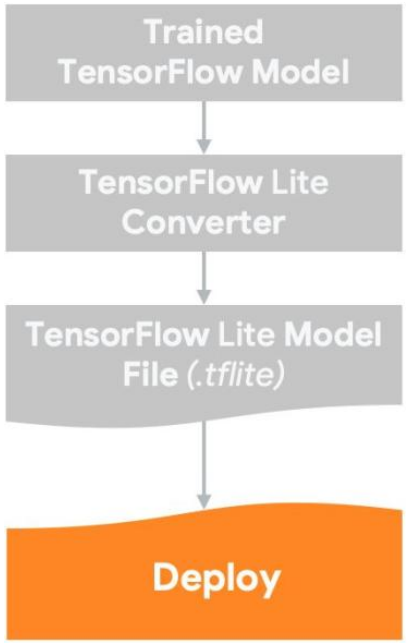
Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

É possível gerar um modelo do TensorFlow Lite das seguintes maneiras:

Convertendo um modelo do TensorFlow em um do TensorFlow Lite: use o ***TensorFlow Lite Converter*** para converter modelos. Durante a conversão, você pode ***aplicar otimizações como a quantização*** para diminuir o tamanho e a latência do modelo com pouca ou nenhuma perda na acurácia. Por padrão, nenhum dos modelos contém metadados.





Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite

Quando se utiliza outros frameworks é necessário converter primeiro para o TensorFlow.



ONNX
Transferência manual dos pesos

O ONNX (Open Neural Network Exchange) é um ***formato aberto*** que padroniza modelos de *machine learning*, permitindo ***interoperabilidade entre frameworks*** como TensorFlow e PyTorch, sem necessidade de reimplementação.

TensorFlow Lite

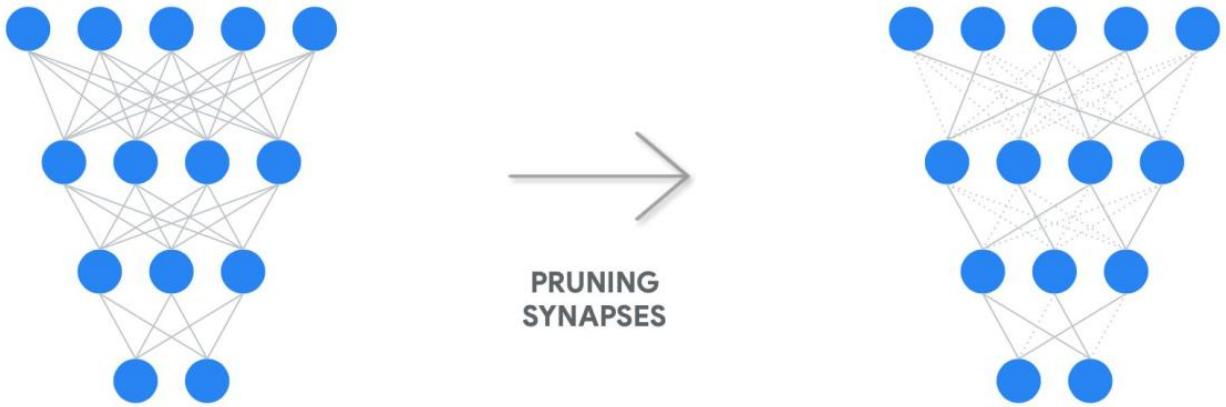
- Como modelos do tensorflow lite rodam em dispositivos com restrições computacionais?
- Aplicando operações de
  - Quantização,
  - Clusterização,
  - Poda,
  - Etc.

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Poda de redes neurais (Pruning)

- Existem diferentes maneiras de podar uma rede neural.
  - (1) Você pode podar pesos. Isso é feito removendo pesos com ***valores absolutos próximos de zero***, tornando a rede ***esparsa***.
  - (2) Você pode remover nós inteiros da rede.
  - Em ambos os casos, (1) e (2), reduz-se o número de parâmetros no modelo.

- (1) é implementado ***removendo-se todos os pesos*** que tenham ***valor absoluto abaixo de um determinado limiar***.



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

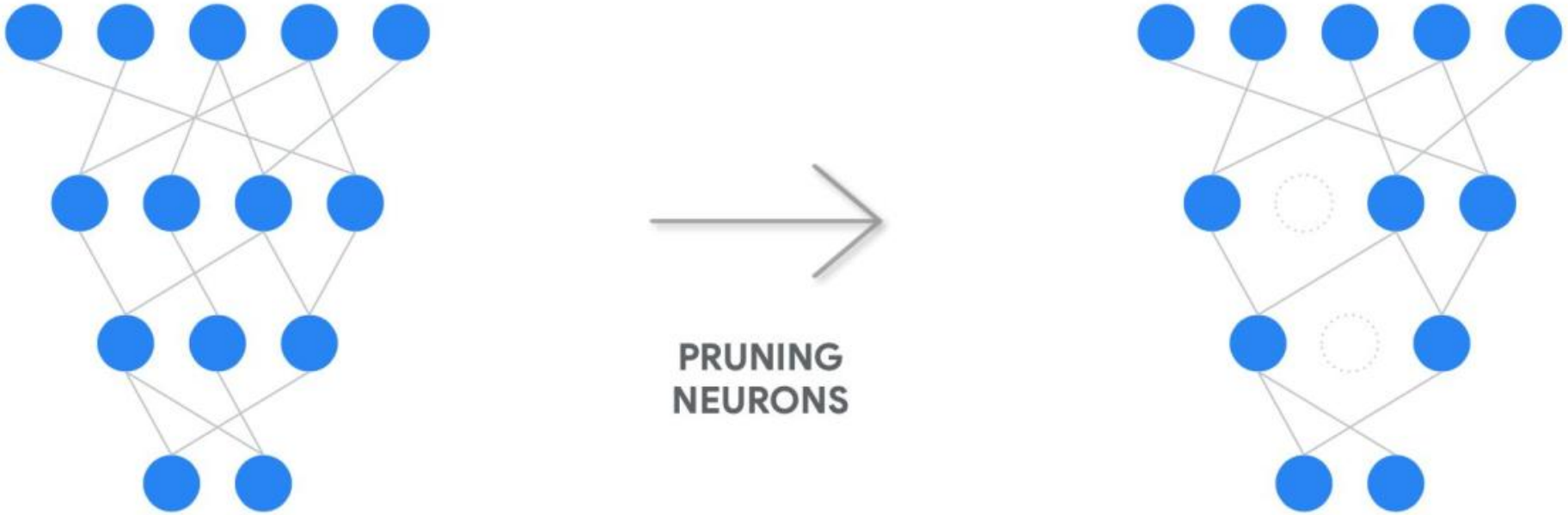
Poda de redes neurais (Pruning)

- Ao invés de remover apenas pesos, as ativações durante o treinamento podem ser usadas como critério para remoção.
- Em (2), ao apresentar um conjunto de dados à rede, certas estatísticas das ativações podem ser observadas.
- Você pode observar que alguns neurônios sempre produzem valores próximos de zero.

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

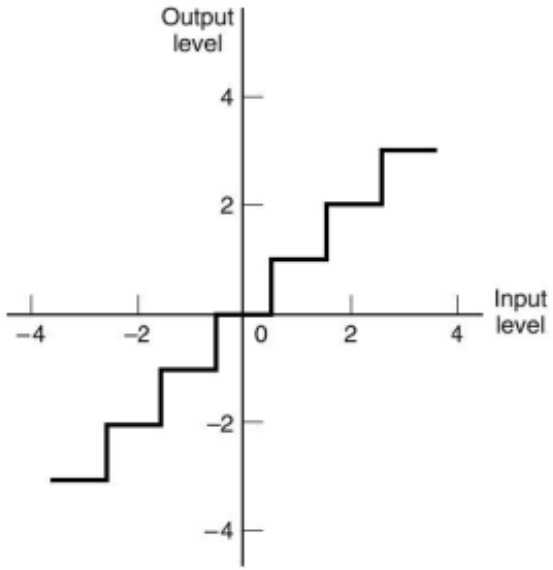
Poda de redes neurais (Pruning)

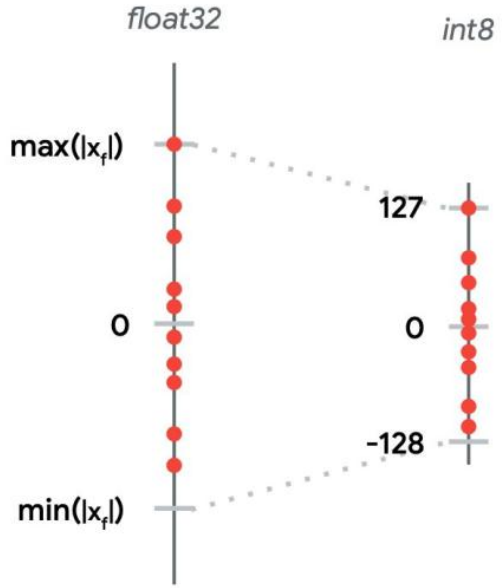
- Se dois neurônios em uma mesma camada tiverem ***pesos muito semelhantes***, isso pode significar que eles estão fazendo a mesma coisa.
- Assim, a ***redundância de pesos*** pode significar que ***um neurônio pode ser removido***.



Quantização

- A quantização é uma otimização que funciona ***reduzindo a precisão*** dos números usados para representar os parâmetros de um modelo, que por padrão são números em ponto flutuante de 32 bits (4 bytes).





Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Quantização

- A operação de quantização não é reversível, i.e., ***perde-se informação***.
- Imagem Lena quantizada em 16, 8, 4 e 2 bits/pixel



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Quantização

- Exemplo de perda de precisão pela quantização.

Modelos usados para classificação de imagens	Floating-point Baseline	Post-training Quantization (PTQ)	Accuracy Drop	Tamanho antes da quantização
MobileNet v1 1.0 224	**71.03%**	**69.57%**	**▼1.46%**	16 MB
MobileNet v2 1.0 224	**70.77%**	**70.20%**	**▼0.57%**	14 MB
Resnet v1 50	**76.30%**	**75.95%**	**▼0.35%**	98 MB

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TensorFlow Lite Micro

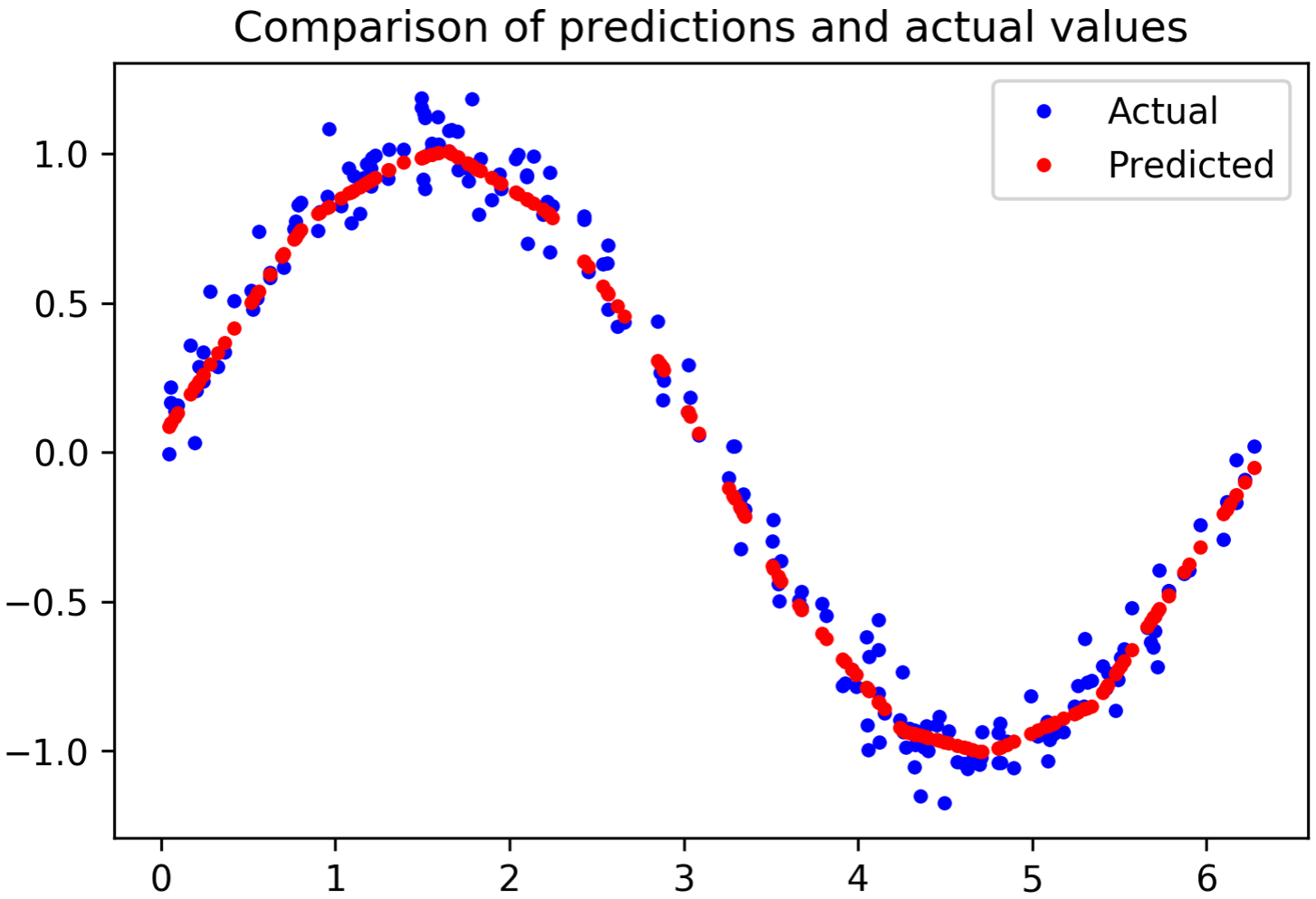
- O TensorFlow Lite for Microcontrollers (TFLM, abreviadamente) foi projetado para executar modelos de aprendizado de máquina em microcontroladores e outros ***dispositivos com grandes restrições computacionais***.
- Ao contrário de processadores móveis de alto desempenho (como a série Cortex-A), os microcontroladores (série Cortex-M ou ESP32) têm restrições computacionais e baixo consumo de energia e são usados em produtos como sensores, geladeiras, máquinas de lavar, etc.

TensorFlow Lite Micro

- O TensorFlow Lite para microcontroladores é ***escrito em C++ 11*** e requer uma plataforma de ***32 bits***.
- Ele foi testado extensivamente em diversos processadores e microcontroladores baseados na arquitetura ARM Cortex-M e foi portado para outras arquiteturas, incluindo ESP32, RISC-V e Hexagon.
- O framework também está disponível como uma ***biblioteca para o Arduino***.
- Ele também pode gerar projetos para ambientes de desenvolvimento como o Mbed.
  - Mbed é uma ***plataforma de desenvolvimento*** e ***sistema operacional*** para dispositivos IoT baseados em microcontroladores ARM Cortex-M de 32 bits.
- Ele é de código aberto e pode ser incluído em qualquer projeto C++ 11.

Exemplo

- Hello world usando o seno de um ângulo $H\tilde(x) = \sin(x)$



Encontrar um modelo que aproxime o comportamento por trás das amostras ruidosas.

Exemplo: Hello world for microcontrollers

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Importação de bibliotecas e definição das sementes

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import math

plt.rcParams['figure.figsize'] = (16, 10)
plt.rc('font', size=15)

# define random seed for reproducibility
np.random.seed(1) # numpy seed
tf.random.set_seed(1) # tensorflow global random seed
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Geração da onda senoidal

```
# Generate a uniformly distributed set of random numbers in the range from
# 0 to 2π, which covers a complete sine wave oscillation
X = np.random.uniform(
    low=0, high=2*math.pi, size=10000).astype(np.float32)

# Shuffle the values to guarantee they're not in order
np.random.shuffle(X)

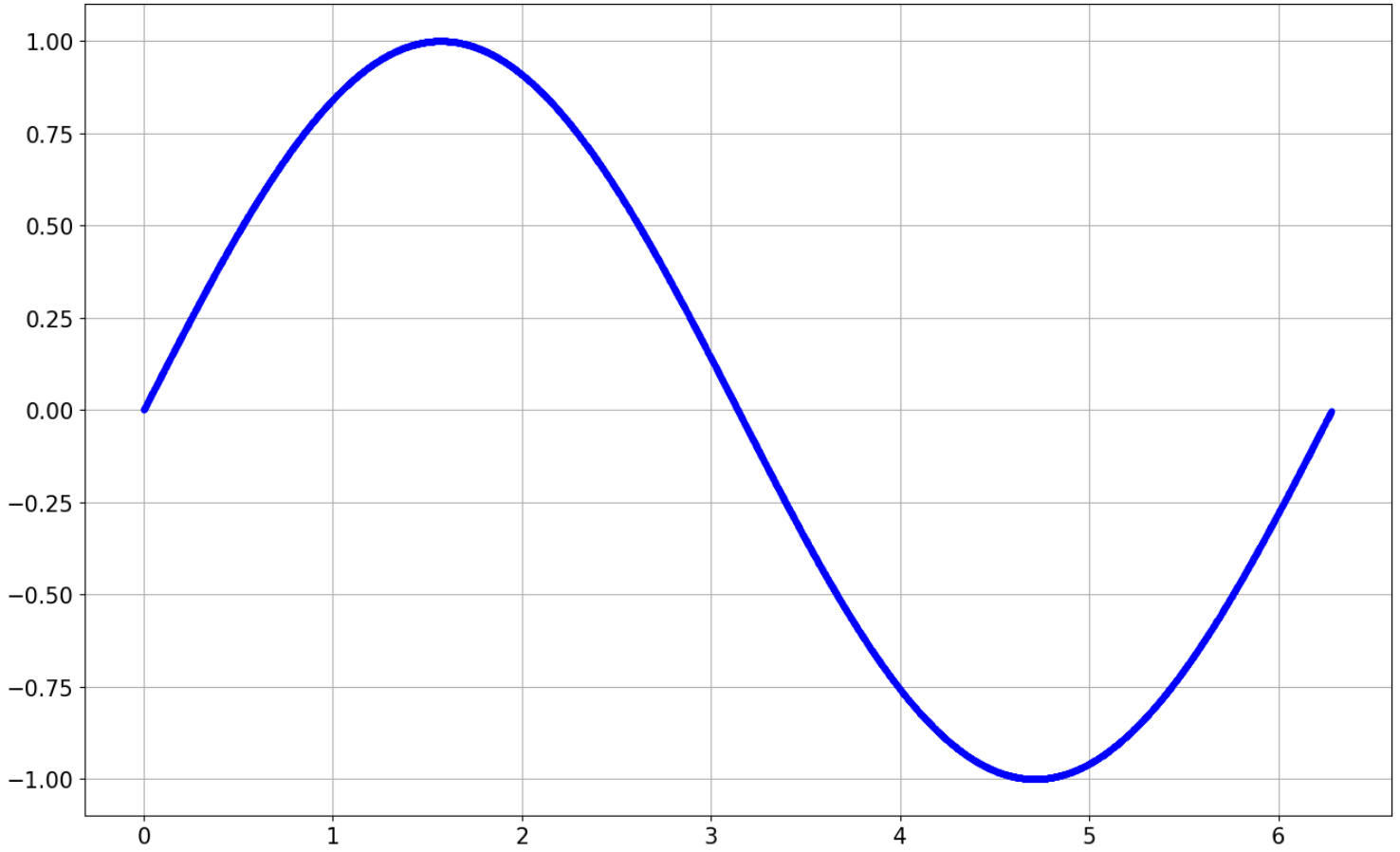
# Calculate the corresponding sine values
y = np.sin(X).astype(np.float32)

# Plot our data. The 'b.' argument tells the library to print blue dots.
plt.plot(X, y, 'b.')
plt.grid()
plt.show()
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Geração da onda senoidal



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Adição de ruído

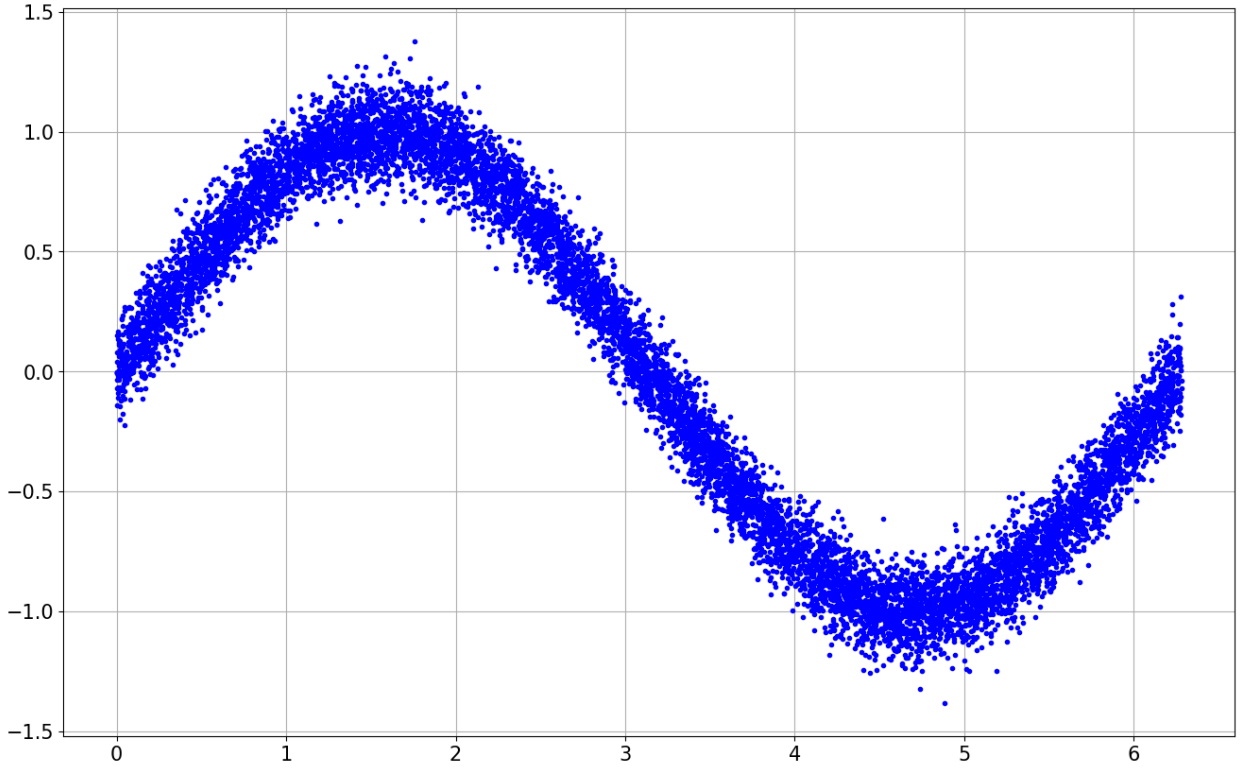
```
[5] # Add a small random number to each y value
    y += 0.1 * np.random.randn(*y.shape)

    # Plot our data
    plt.plot(X, y, 'b.')
    plt.grid()
    plt.show()
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(x)} = \sin(x)$
- Adição de ruído



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(x)} = \sin(x)$
- Separação das amostras em treinamento, validação e teste

- Train data: 60%
- Validation data: 20%
- Test data: 20%

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
X_train, X_val, y_train, y_val = train_test_split(X_train, y_train, test_size=0.25, random_state=1)

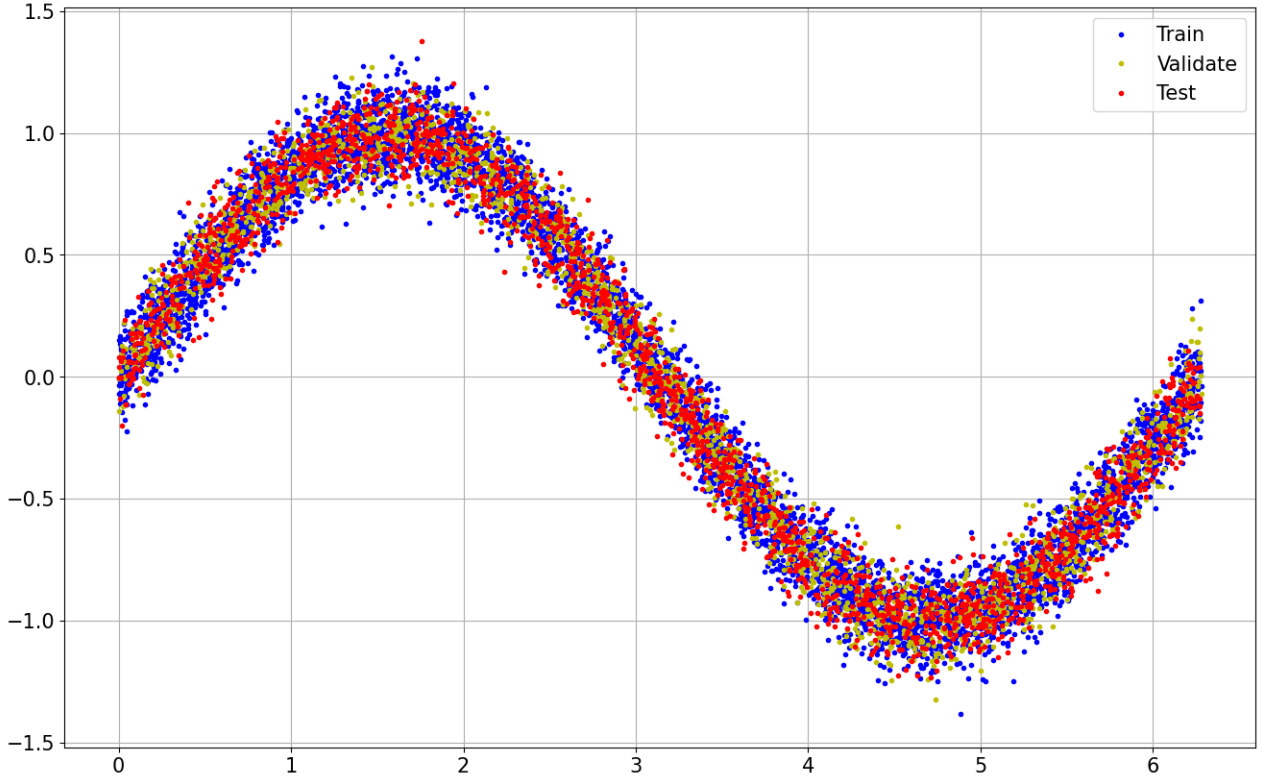
# Plot the data in each partition in different colors:
plt.plot(X_train, y_train, 'b.', label="Train")
plt.plot(X_val, y_val, 'y.', label="Validate")
plt.plot(X_test, y_test, 'r.', label="Test")
plt.legend()
plt.grid()
plt.show()
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H(\tilde{x}) = \sin(x)$
- Separação das amostras em treinamento, validação e teste

- Train data: 60%
- Validation data: 20%
- Test data: 20%



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Modelo inicial

```
model = tf.keras.Sequential(name='sine')
model.add(tf.keras.layers.Dense(8, activation='relu', input_shape=(1, )))
model.add(tf.keras.layers.Dense(1))

model.summary()
```

```
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 8)	16
dense_1 (Dense)	(None, 1)	9

```
Total params: 25 (100.00 B)
Trainable params: 25 (100.00 B)
Non-trainable params: 0 (0.00 B)
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Treinamento

```
history = model.fit(X_train, y_train, epochs=500, batch_size=64, validation_data=(X_val, y_val))
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Gráfico da métrica loss para treinamento e validação

```
# Draw a graph of the loss, which is the distance between
# the predicted and actual values during training and validation.
loss = history.history['loss']
val_loss = history.history['val_loss']

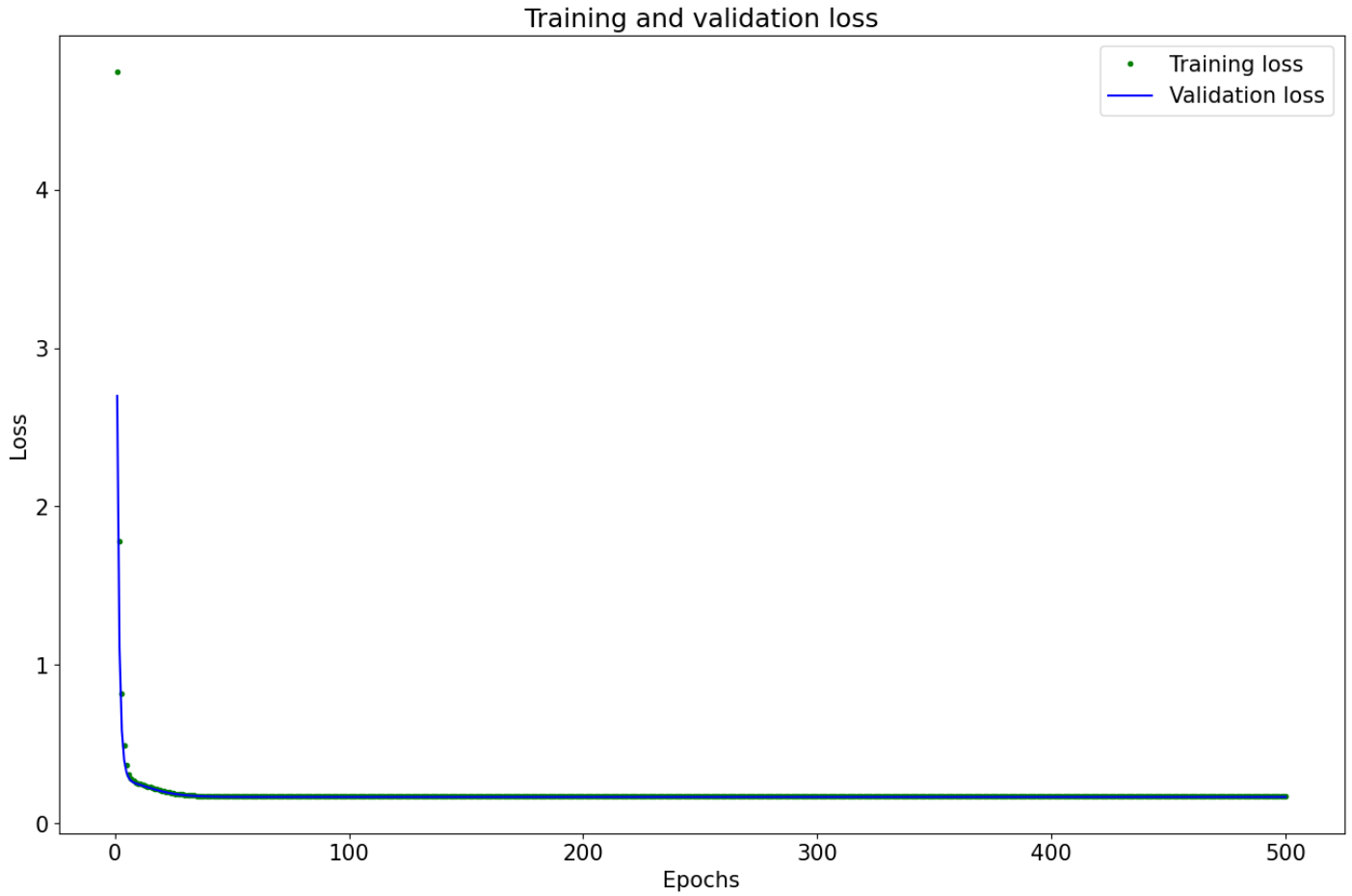
epochs = range(1, len(loss) + 1)

plt.plot(epochs, loss, 'g.', label='Training loss')
plt.plot(epochs, val_loss, 'b', label='Validation loss')
plt.title('Training and validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Gráfico da métrica loss para treinamento e validação



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Cálculo do loss e predição do modelo para o dataset de test

```
# Calculate and print the loss on our test dataset
loss = model.evaluate(X_test, y_test)

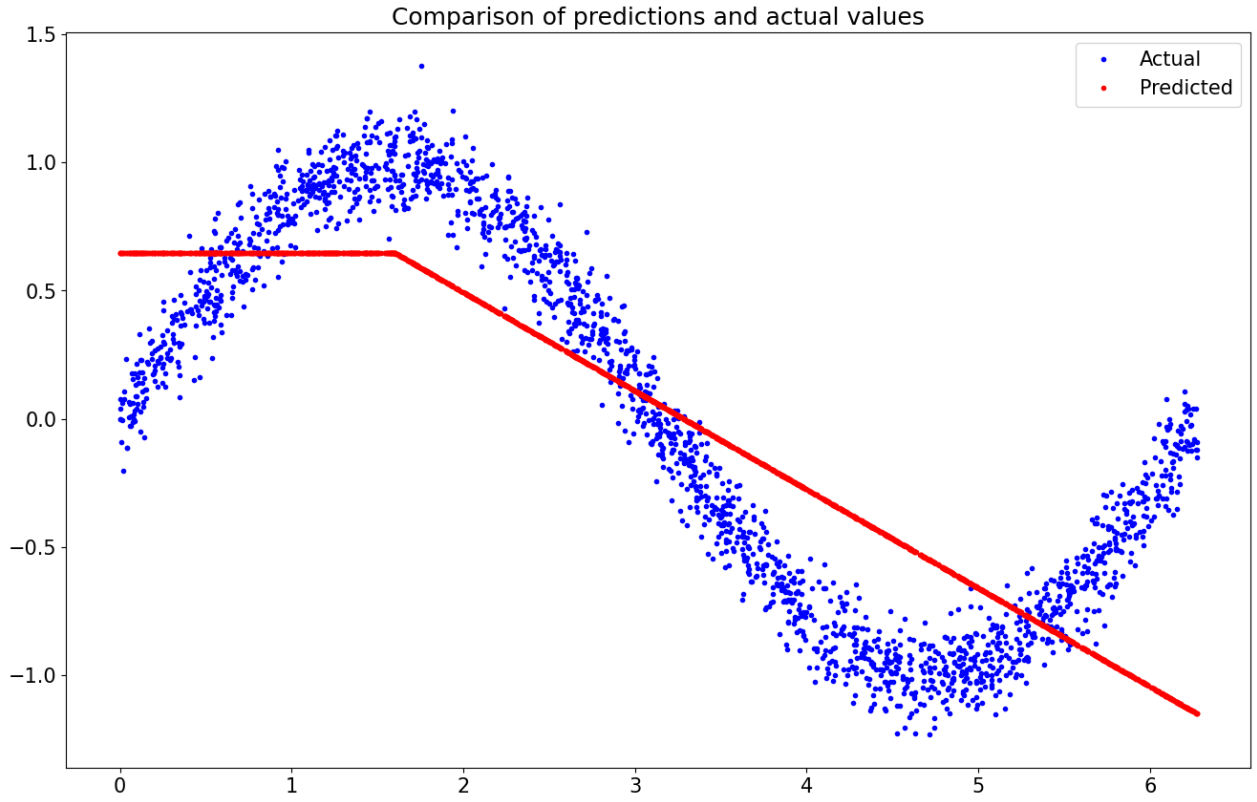
# Make predictions based on our test dataset
predictions = model.predict(X_test)

# Graph the predictions against the actual values
plt.clf()
plt.title('Comparison of predictions and actual values')
plt.plot(X_test, y_test, 'b.', label='Actual')
plt.plot(X_test, predictions, 'r.', label='Predicted')
plt.legend()
plt.show()
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H(\tilde{x}) = \sin(x)$
- Predição do modelo para o dataset de teste



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Segundo modelo com uma camada densa extra e mais neurônios na primeira camada.

```
model_2 = tf.keras.Sequential(name='sine2')

# First layer takes a scalar input and feeds it through 16 "neurons". The
# neurons decide whether to activate based on the 'relu' activation function.
model_2.add(tf.keras.layers.Dense(16, activation='relu', input_shape=(1,)))

# The new second layer may help the network learn more complex representations
model_2.add(tf.keras.layers.Dense(16, activation='relu'))

# Final layer is a single neuron, since we want to output a single value
model_2.add(tf.keras.layers.Dense(1))

# Compile the model using a standard optimizer and loss function for regression
model_2.compile(optimizer='adam', loss='mse', metrics=['mae'])

model_2.summary()
```

```
history = model_2.fit(X_train, y_train, epochs=500, batch_size=64, validation_data=(X_val, y_val))
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

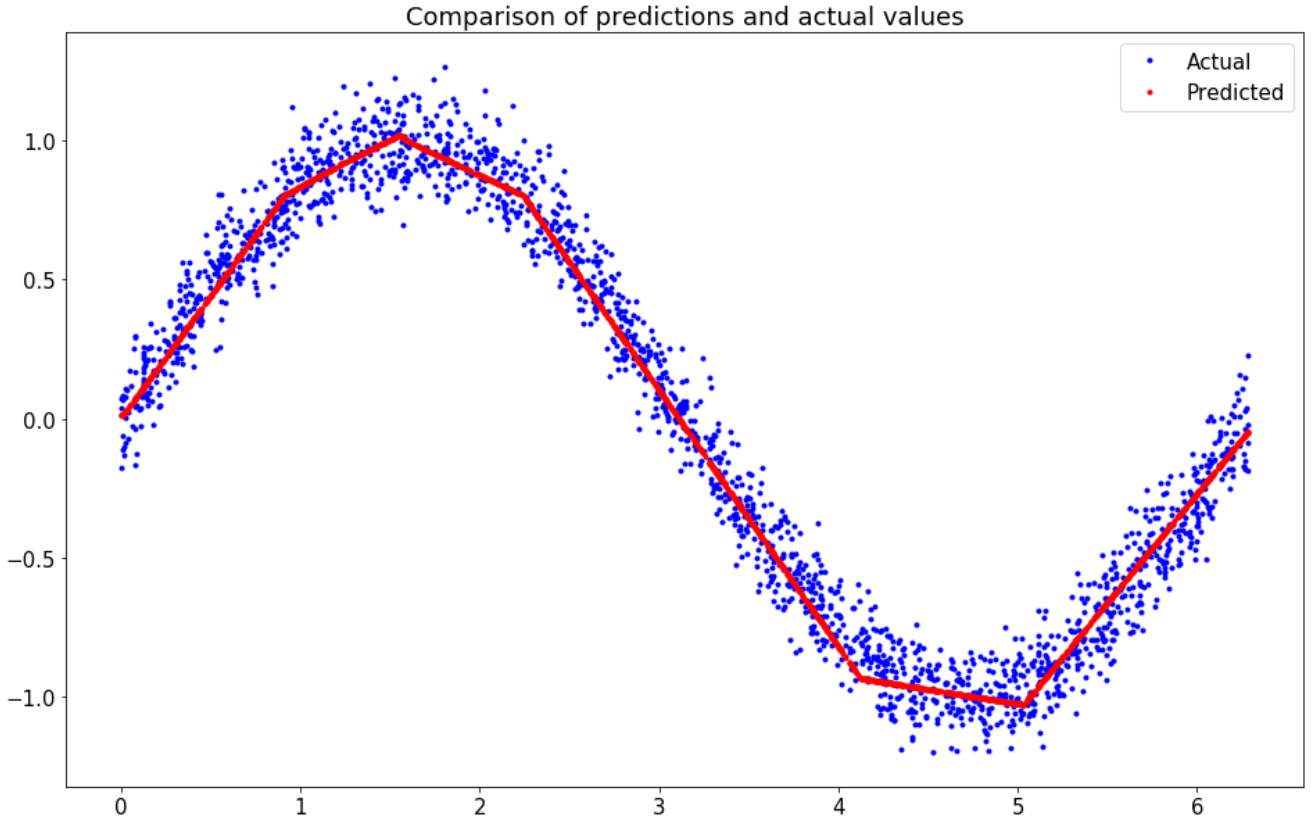
Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Segundo modelo com uma camada densa extra e mais neurônios na primeira camada.

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 16)	32
dense_3 (Dense)	(None, 16)	272
dense_4 (Dense)	(None, 1)	17

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$
- Predição do modelo para o dataset de teste



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H(\tilde{x}) = \sin(x)$

Conversão dos modelos

Criação de um diretório para os modelos

```
import os
MODEL_DIR = './models/'
if not os.path.exists(MODEL_DIR):
    os.mkdir(MODEL_DIR)
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H(\tilde{x}) = \sin(x)$

Conversão dos modelos

```
# Convert the model to the TensorFlow Lite format without quantization
converter = tf.lite.TFLiteConverter.from_keras_model(model_2)
model_no_quant_tflite = converter.convert()

# # Save the model to disk
open(MODEL_DIR + 'model_no_quant.tflite', "wb").write(model_no_quant_tflite)
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$

Conversão dos modelos: Quantização total do modelo, com exceção das entradas e saídas, para manter a interface original.

```
# Convert the model to the TensorFlow Lite format with quantization
def representative_dataset():
    for i in range(500):
        yield([X_train[i].reshape(1, 1)])
# Set the optimization flag.
converter.optimizations = [tf.lite.Optimize.DEFAULT]
# Enforce full-int8 quantization (except inputs/outputs which are always float)
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
# Provide a representative dataset to ensure we quantize correctly.
converter.representative_dataset = representative_dataset
model_tflite = converter.convert()

# Save the model to disk
open(MODEL_DIR + 'model.tflite', "wb").write(model_tflite)
```

Full integer quantization: usa amostras para calibrar os intervalos dos tensores.

*representative_dataset(): Essa função fornece exemplos de dados de entrada para que o conversor do TFLite calibre os intervalos dinâmicos dos tensores do modelo, otimizando-o para tamanho e velocidade.

https://ai.google.dev/edge/litert/models/post_training_quantization

Exemplo

- Hello world seno de um ângulo $H(\tilde{x}) = \sin(x)$

Conversão dos modelos

Comparação dos tamanhos dos modelos

```
model_original_size = os.path.getsize(MODEL_DIR + 'model_2_original.h5')
print("Original model is %d bytes" % model_original_size)

model_no_quant_size = os.path.getsize(MODEL_DIR + 'model_2_no_quant.tflite')
print("Non-quantized model is %d bytes" % model_no_quant_size)

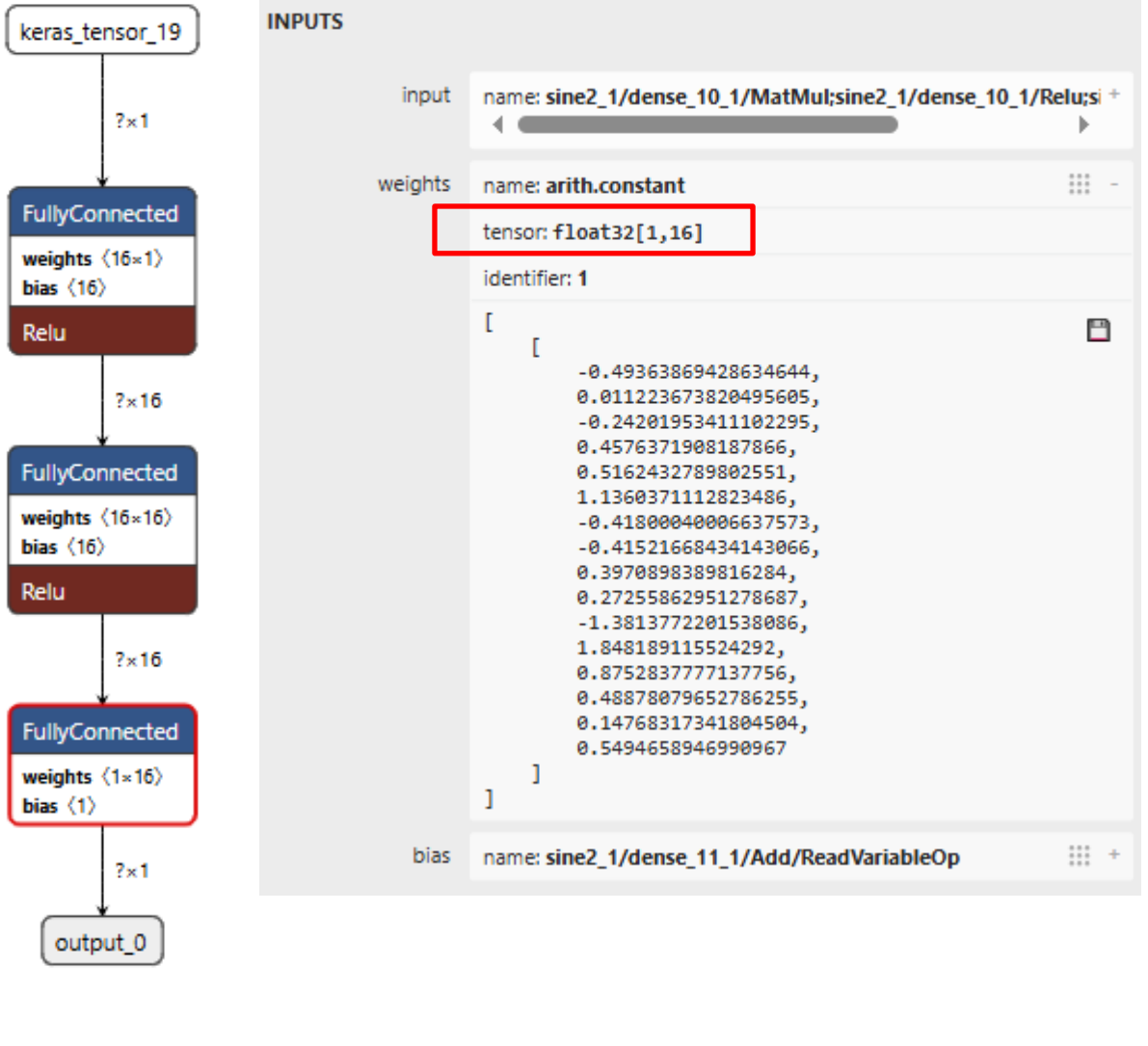
model_quant_size = os.path.getsize(MODEL_DIR + 'model_2_quant.tflite')
print("Quantized model is %d bytes" % model_quant_size)

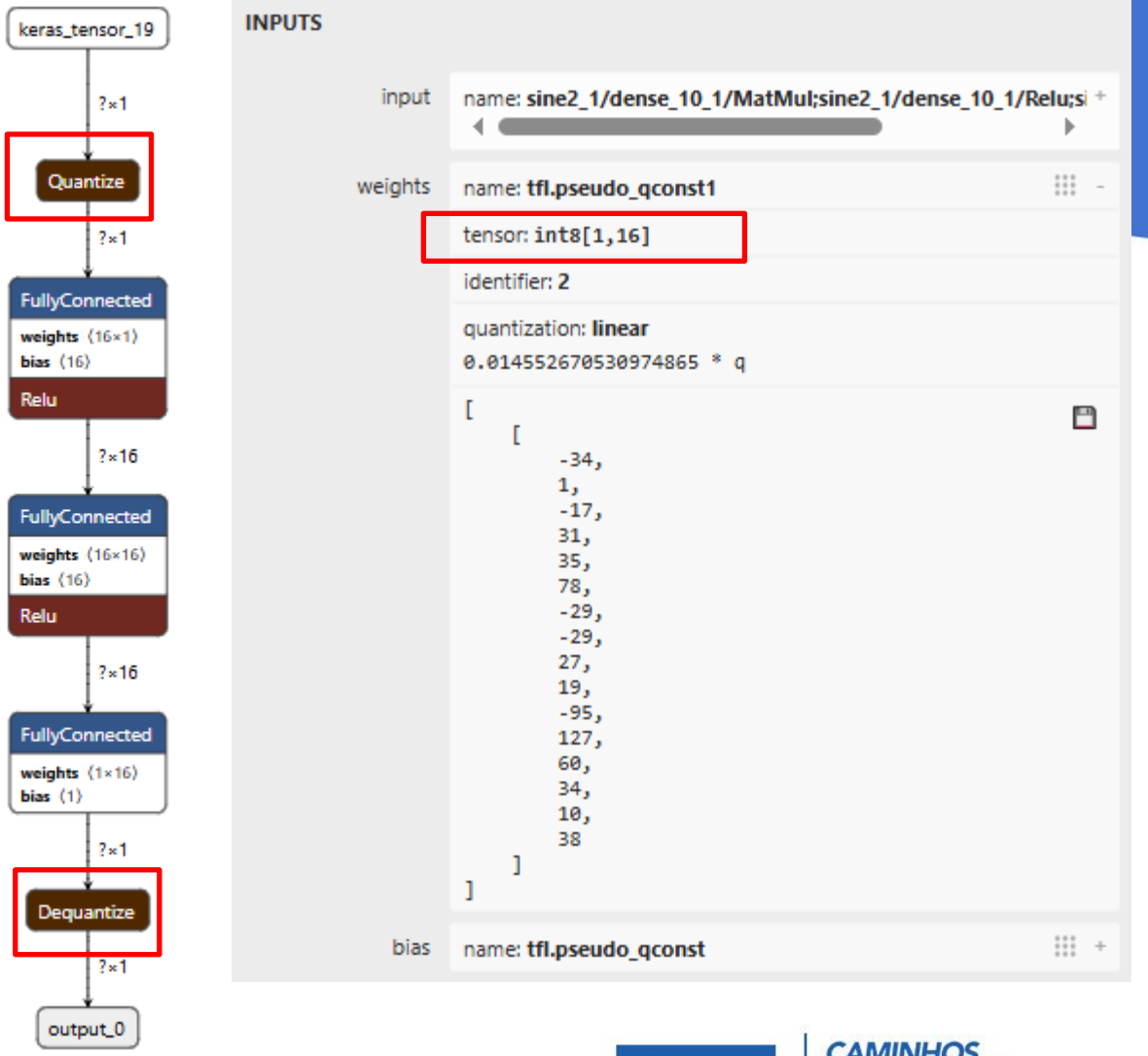
difference = model_no_quant_size - model_quant_size
print("Difference is %d bytes" % difference)
```

```
Original model is 30984 bytes
Non-quantized model is 3116 bytes
Quantized model is 3744 bytes
Difference is -628 bytes
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo





https://netron.app/

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H\tilde{(}x) = \sin(x)$

Conversão dos modelos

Fazendo a predição para os modelos criados

```
# Instantiate an interpreter for each model
model_no_quant = tf.lite.Interpreter(MODEL_DIR + 'model_no_quant.tflite')
model = tf.lite.Interpreter(MODEL_DIR + 'model.tflite')

# Allocate memory for each model
model_no_quant.allocate_tensors()
model.allocate_tensors()

# Get the input and output tensors so we can feed in values and get the results
model_no_quant_input = model_no_quant.tensor(model_no_quant.get_input_details()[0]["index"])
model_no_quant_output = model_no_quant.tensor(model_no_quant.get_output_details()[0]["index"])
model_input = model.tensor(model.get_input_details()[0]["index"])
model_output = model.tensor(model.get_output_details()[0]["index"])

# Create arrays to store the results
model_no_quant_predictions = np.empty(X_test.size)
model_predictions = np.empty(X_test.size)

# Run each model's interpreter for each value and store the results in arrays
for i in range(X_test.size):
    model_no_quant_input().fill(X_test[i])
    model_no_quant.invoke()
    model_no_quant_predictions[i] = model_no_quant_output()[0]

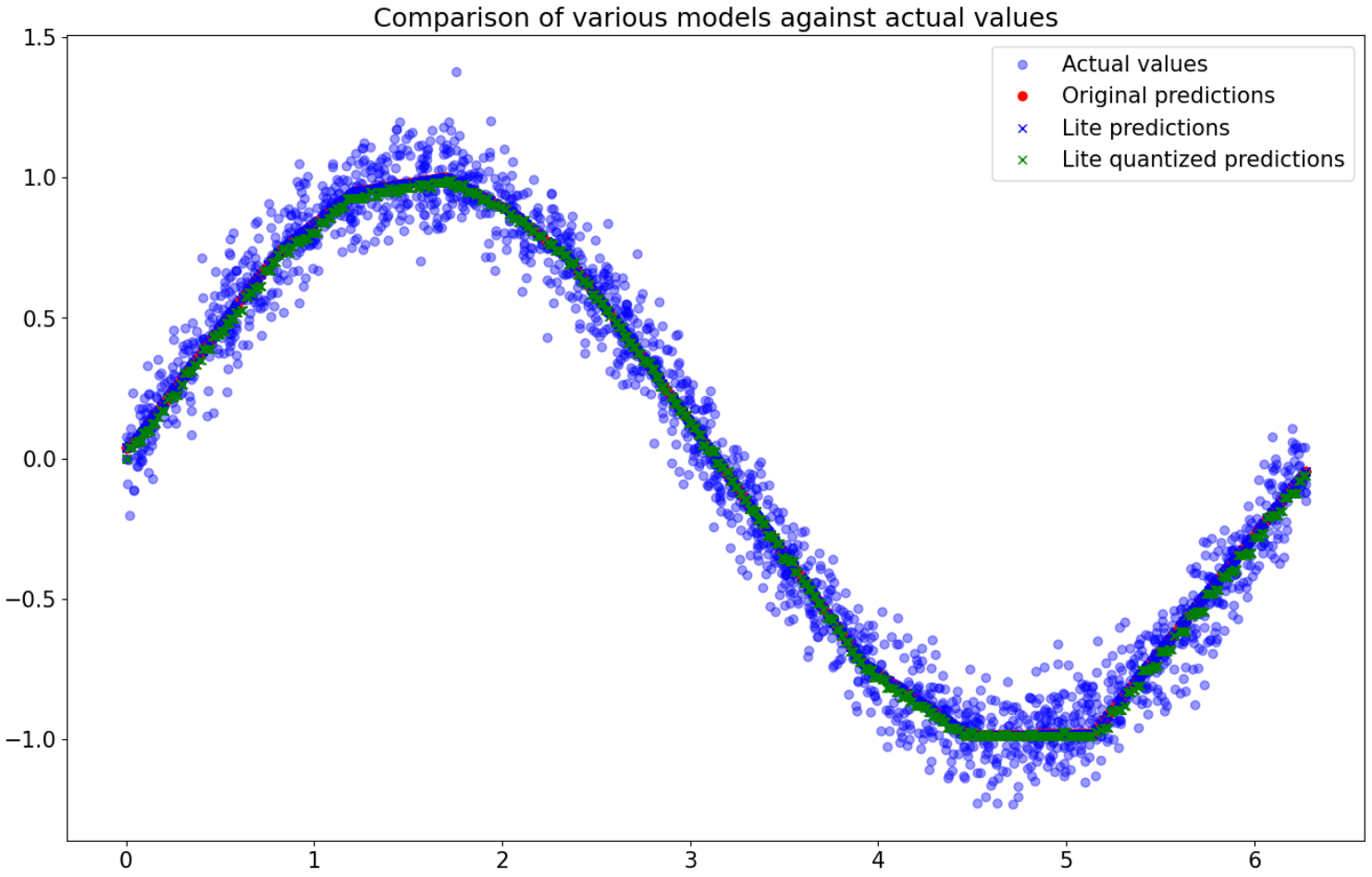
    model_input().fill(X_test[i])
    model.invoke()
    model_predictions[i] = model_output()[0]
```

Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplo

- Hello world seno de um ângulo $H(\tilde{x}) = \sin(x)$

Comparação dos modelos



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

Exemplos de clusterização e poda

- Prunning: https://colab.research.google.com/github/zz4fap/tp557-iot-ml/blob/main/examples/pruning_with_keras.ipynb
- Clusterização: https://colab.research.google.com/github/zz4fap/tp557-iot-ml/blob/main/examples/clustering_example.ipynb

Edge Impulse

- Edge Impulse é uma plataforma para desenvolver e implantar modelos de aprendizado de máquina em dispositivos de borda.
- Ele fornece um conjunto de ferramentas e serviços que facilitam a coleta de dados, o treinamento de modelos e a implantação deles em dispositivos de borda.
- Edge Impulse oferece suporte a uma ampla variedade de dispositivos de borda, incluindo microcontroladores, sistemas embarcados e dispositivos móveis.

Edge Impulse

- O Edge Impulse é usado por uma ampla variedade de empresas e organizações.





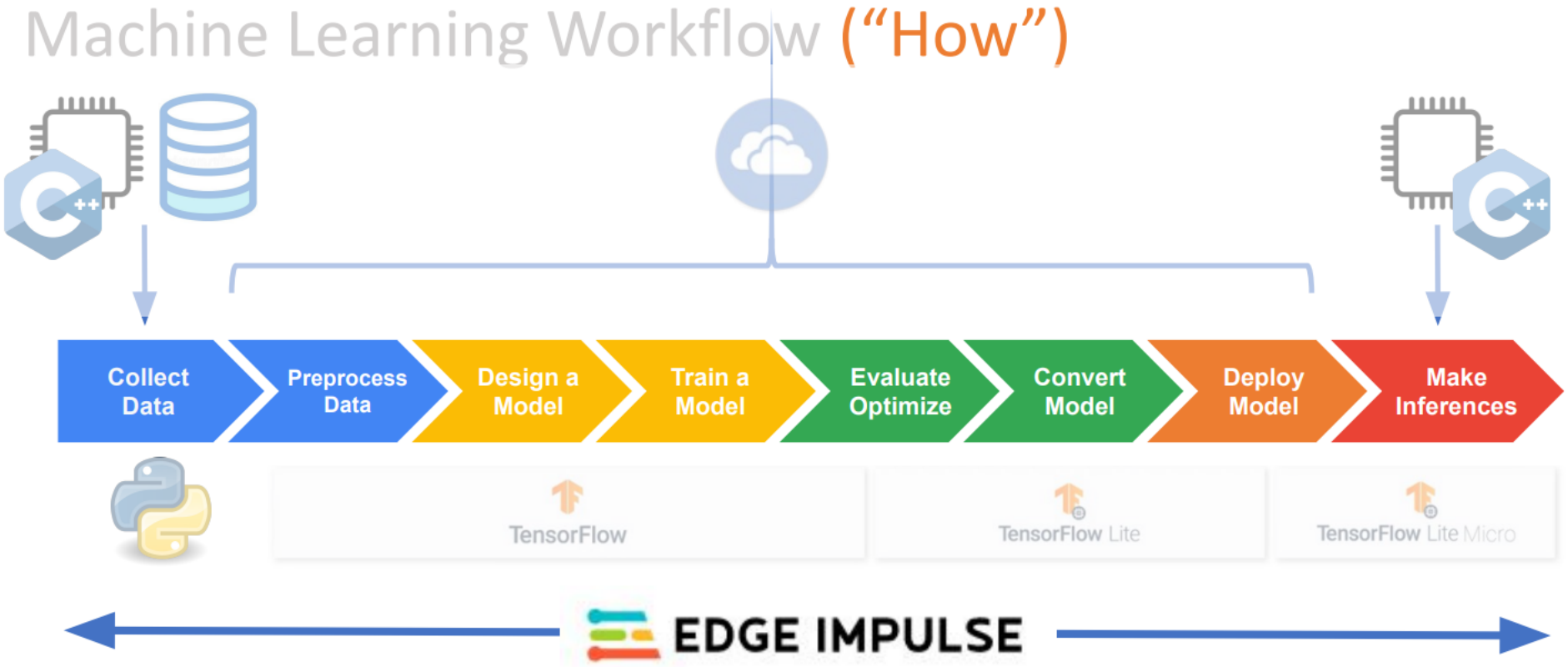








Edge Impulse



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

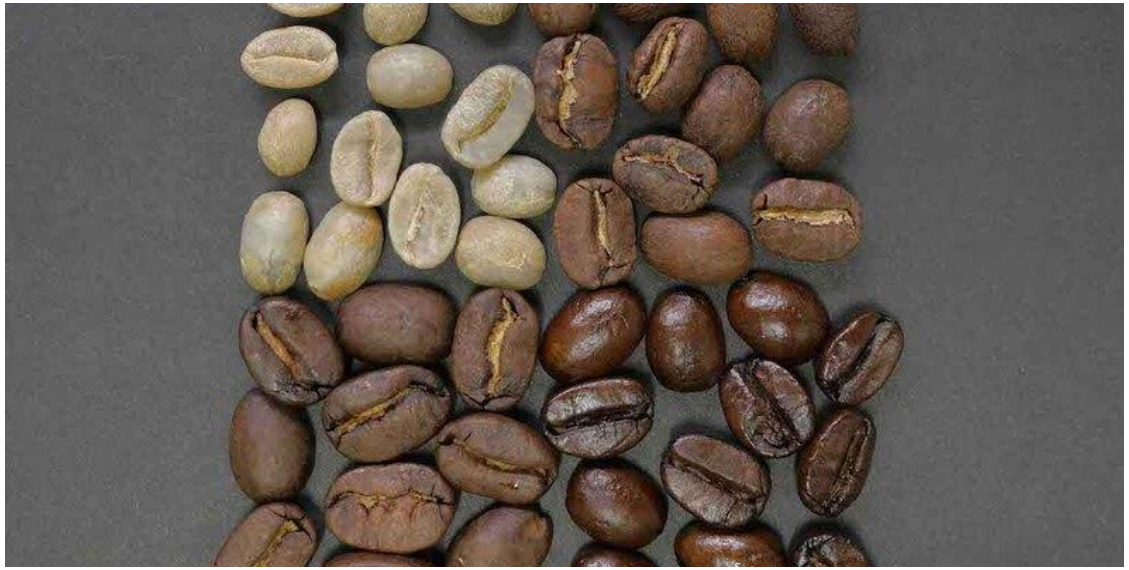
Edge Impulse

- Fácil de usar: O Edge Impulse é fácil de usar, mesmo para desenvolvedores sem experiência anterior em aprendizado de máquina.
- Abrangente: O Edge Impulse fornece um conjunto abrangente de ferramentas e serviços para desenvolver e implantar modelos de aprendizado de máquina em dispositivos de borda.
- Flexível: O Edge Impulse oferece suporte a uma ampla variedade de dispositivos de borda e estruturas de aprendizado de máquina.
- Escalável: O Edge Impulse pode ser usado para desenvolver e implantar modelos de aprendizado de máquina em um grande número de dispositivos de borda.

Edge Impulse

→ Tarefa para casa

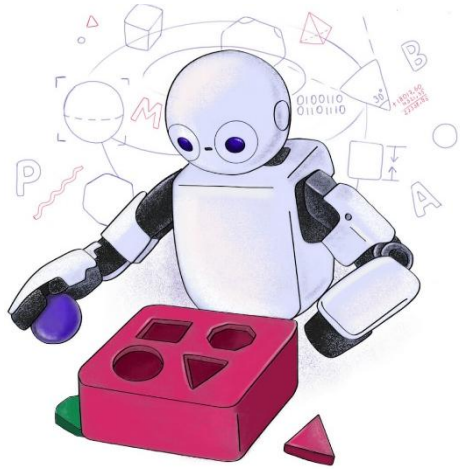
- Criar um dataset ou escolher um do kaggle ou roboflow, treinar um modelo no edge impulse (https://edgeimpulse.com/) e realizar inferências no celular.
  - https://www.kaggle.com/datasets
  - https://universe.roboflow.com/
- Uma opção é usar o dataset para classificação de grãos de café:
  - https://www.kaggle.com/datasets/gpiosenka/coffee-bean-dataset-resized-224-x-224



Inatel
CAMINHOS QUE CONECTAM COM O FUTURO

TP557 - Tópicos avançados em IoT e Machine Learning:

Arduino nano BLE 33 sense

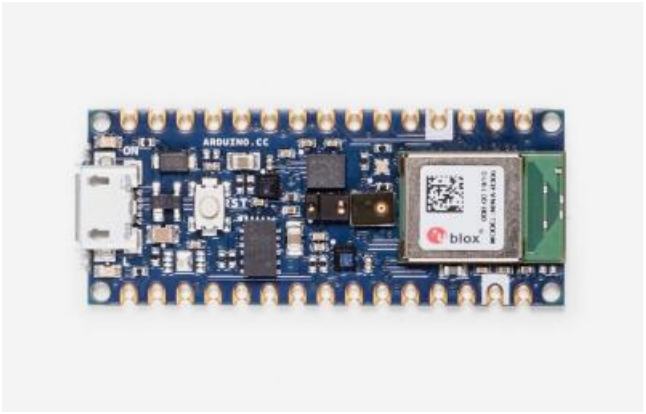


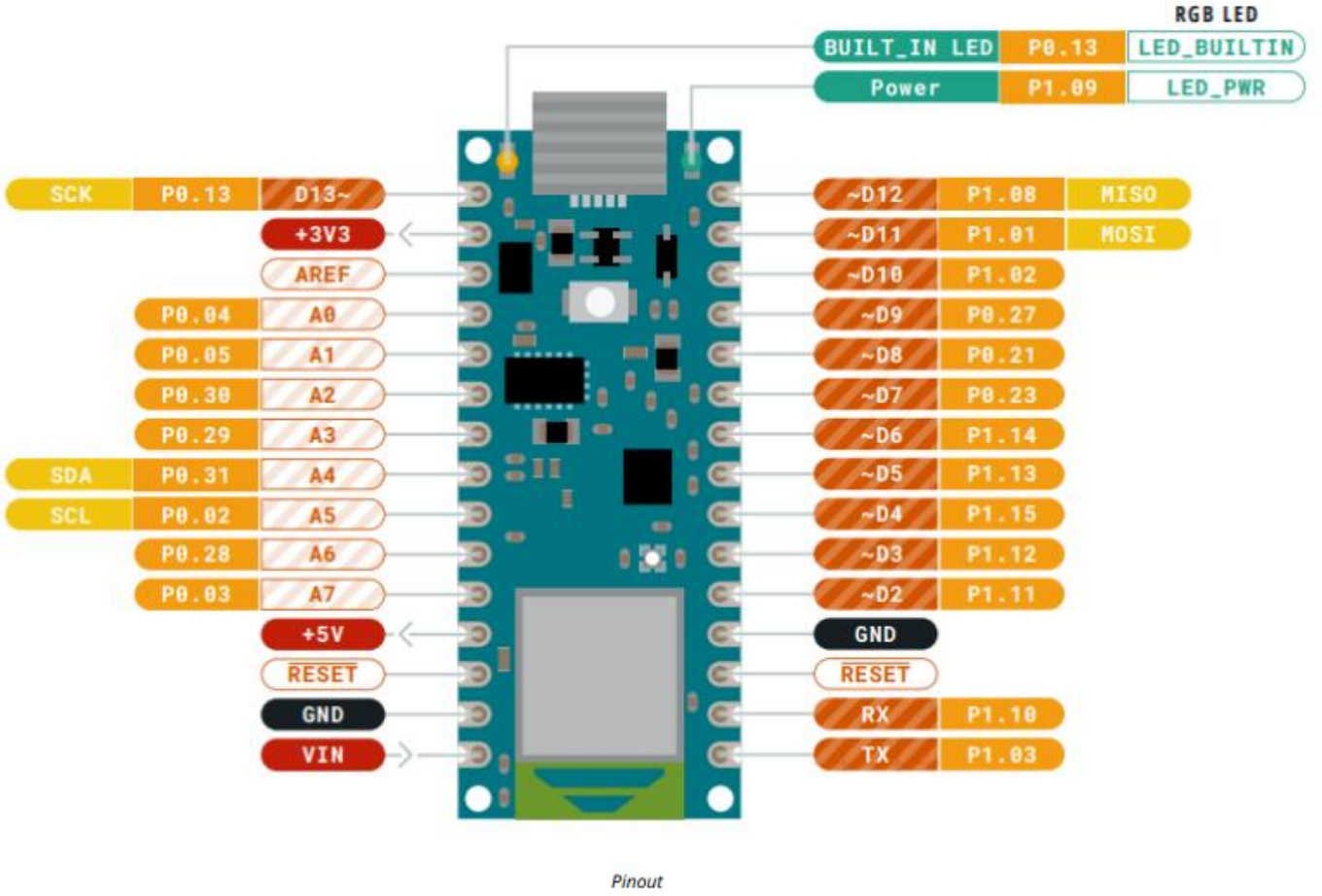
Inatel

Créditos: Prof. Marcelo Rovai UNIFEI

Samuel Baraldi Mafra
samuelbmafra@inatel.br

Arduino nano BLE 33 sense





Pinout

Cada grupo ficará com um kit para realizar os testes durante a aula.

Arduino nano BLE 33 sense

Sensores e protocolo



Bluetooth®



IMU for Motion Detection

unidade de medição inercial (IMU): acelerômetro, giroscópio e magnetômetro



Microphone



Proximity and Gesture Detection



Barometric Pressure Sensor

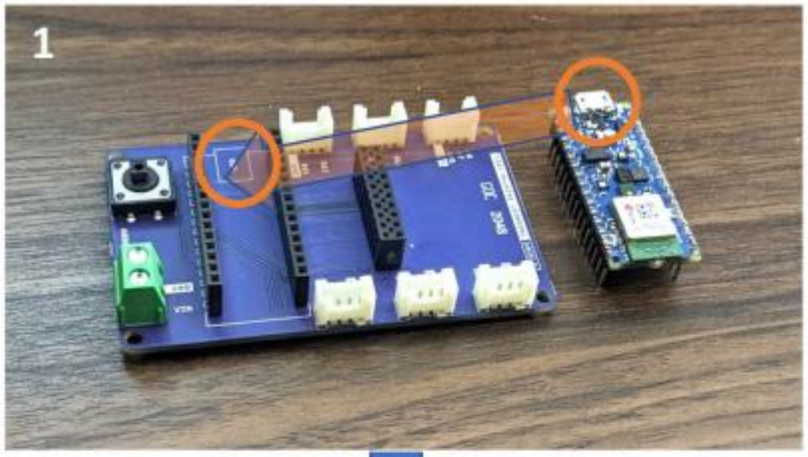


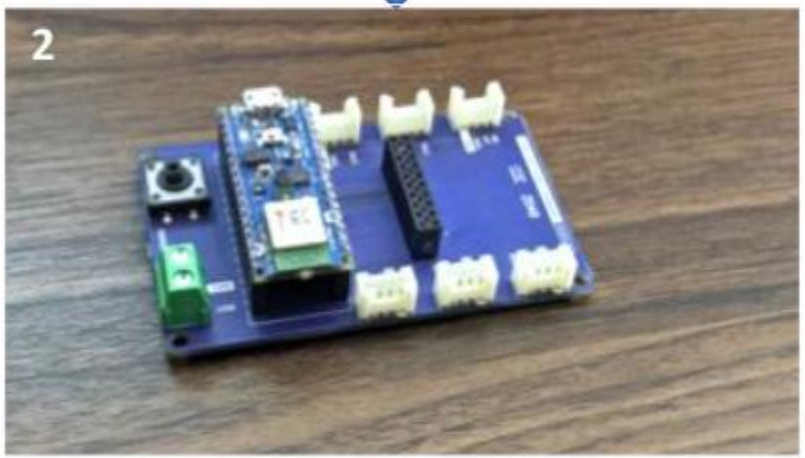
Temperature and *Humidity Sensor

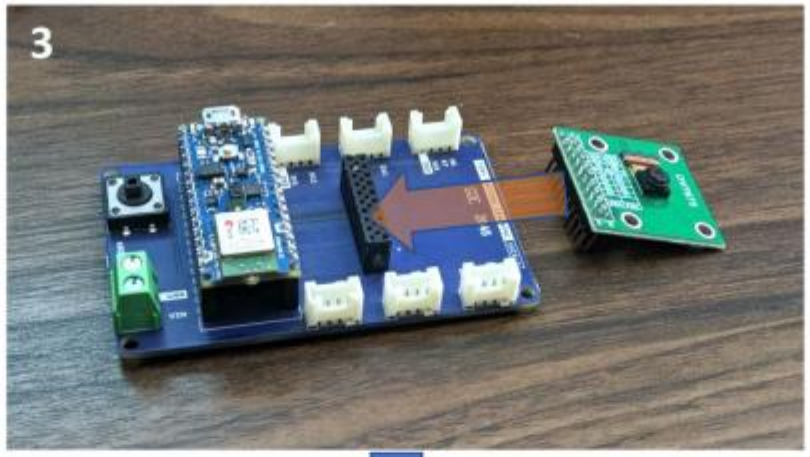
*Não está instalado em algumas versões.

Arduino nano BLE 33 sense

Instalando o hardware





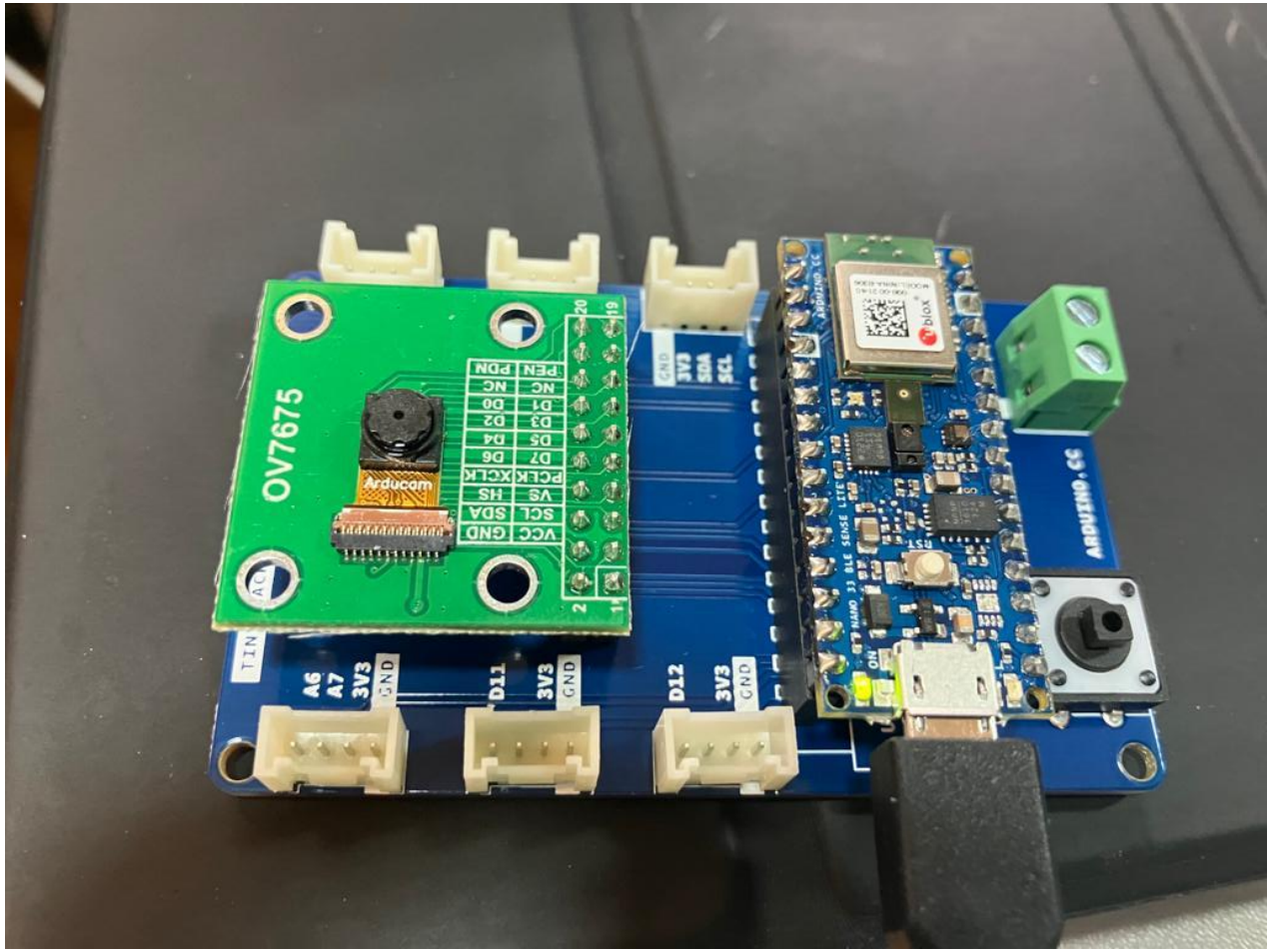






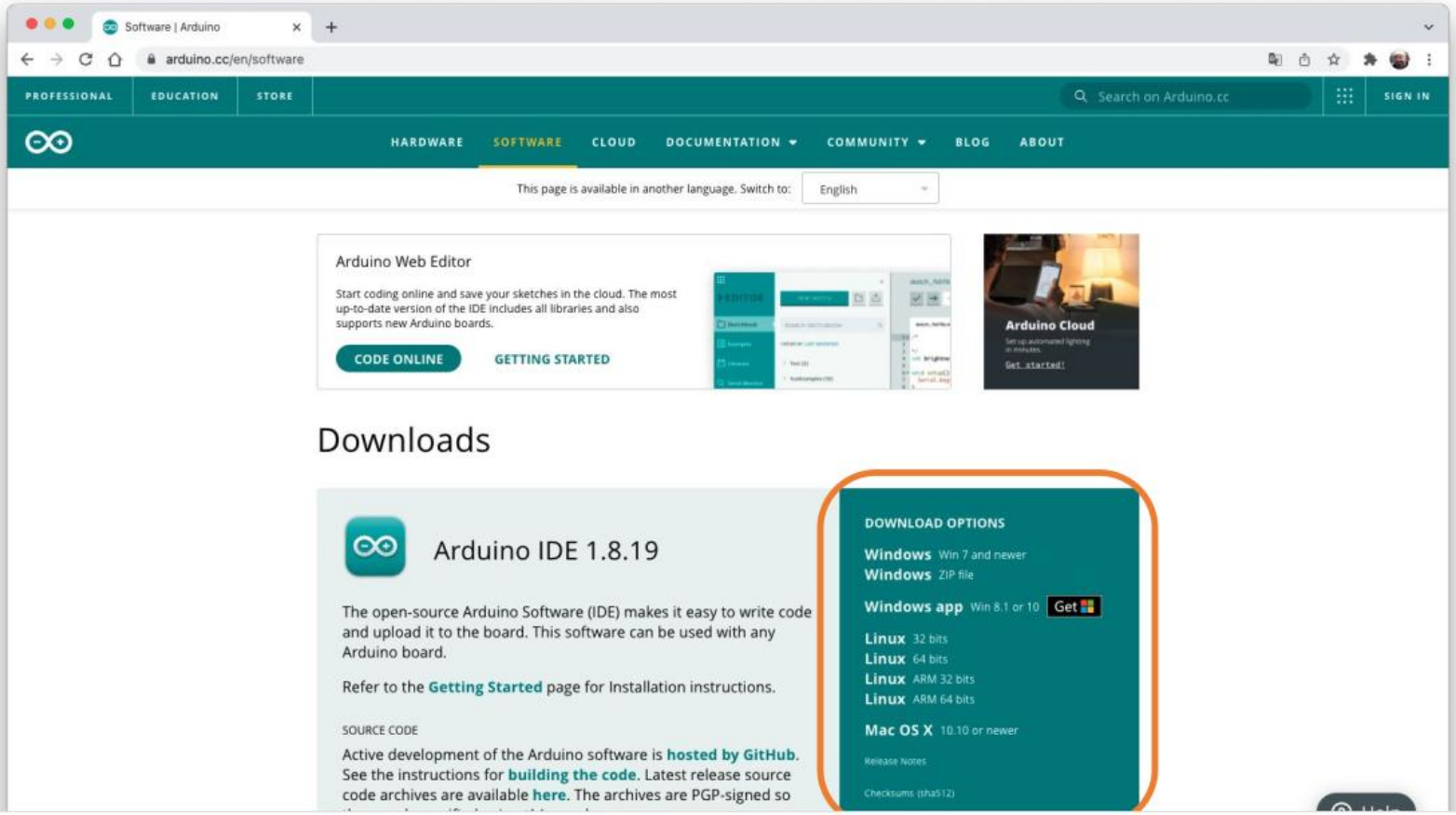


Arduino nano BLE 33 sense



Arduino nano BLE 33 sense

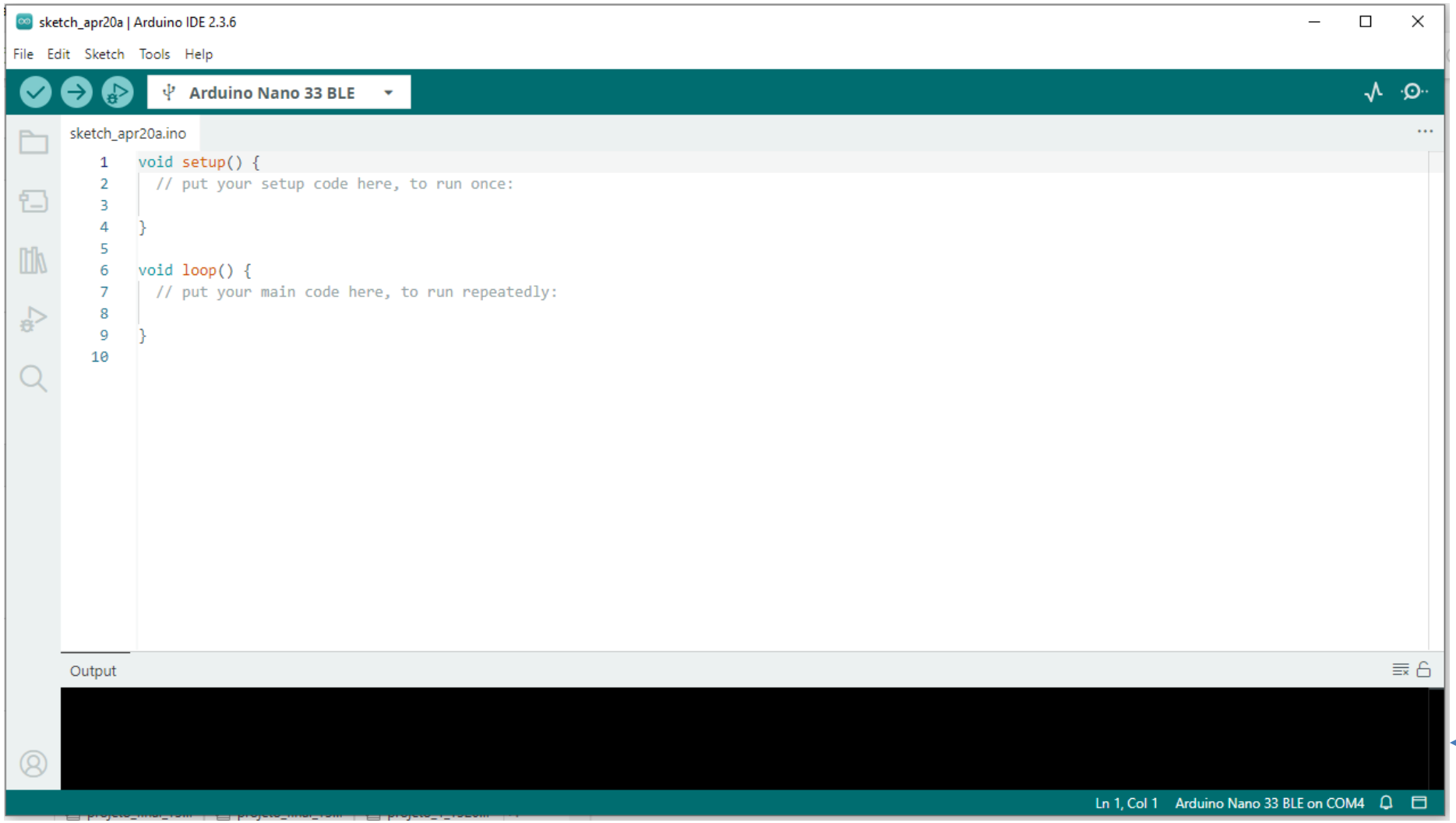
Installing the Arduino IDE



https://www.arduino.cc/en/software/

Arduino nano BLE 33 sense

Menus e barra de tarefas

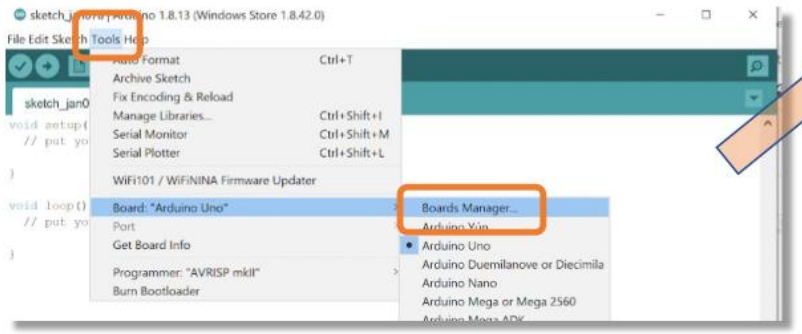


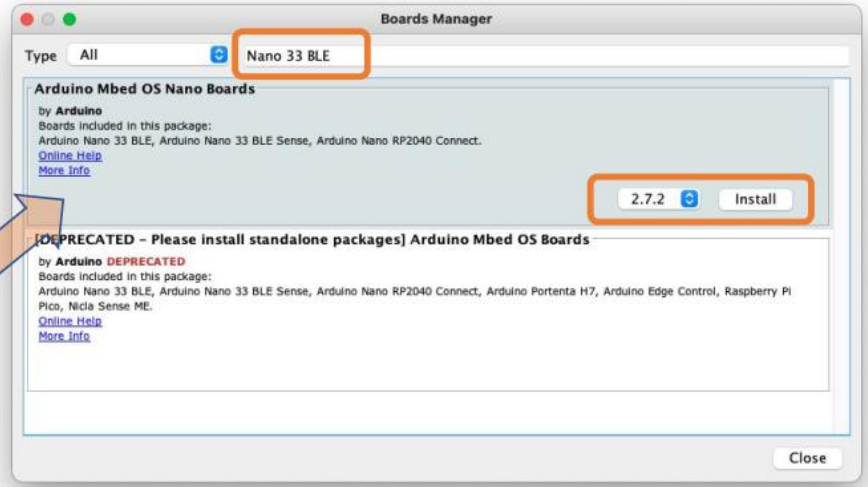
Área de código

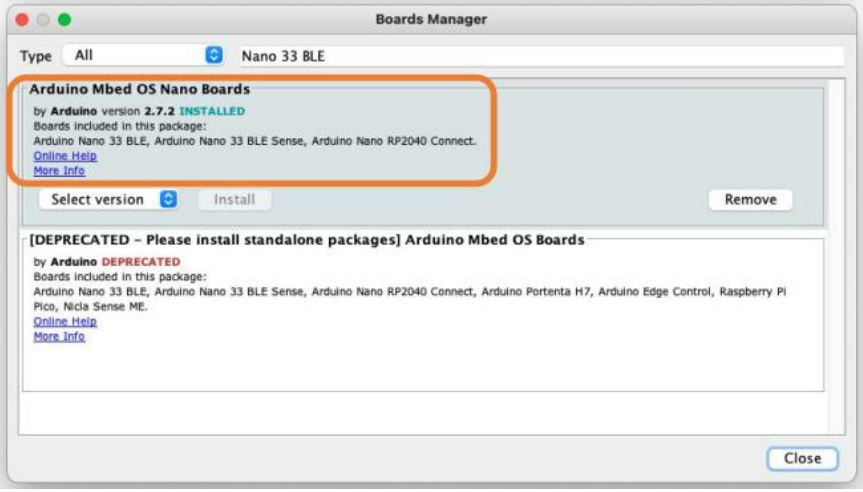
Console

Arduino nano BLE 33 sense

Installing the Board Files

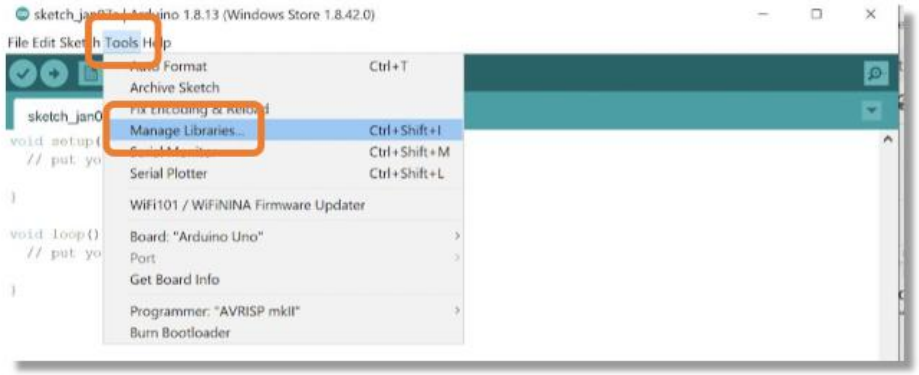


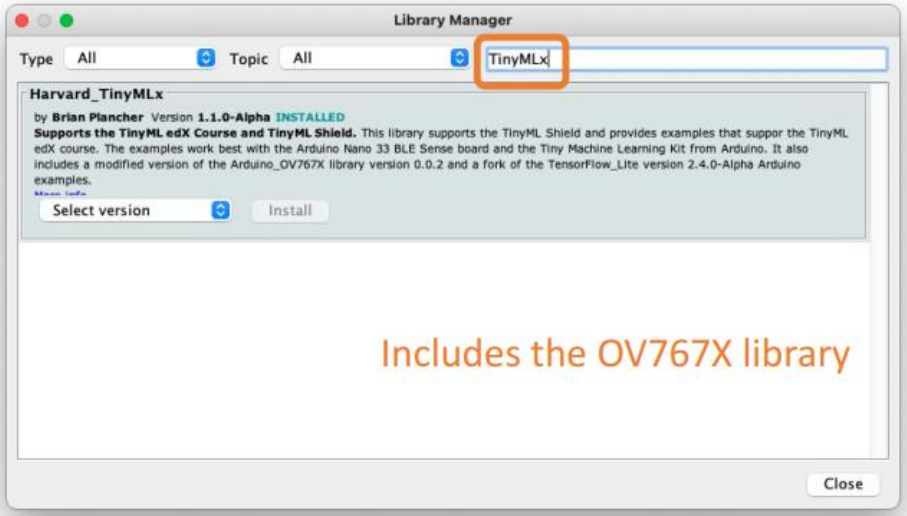


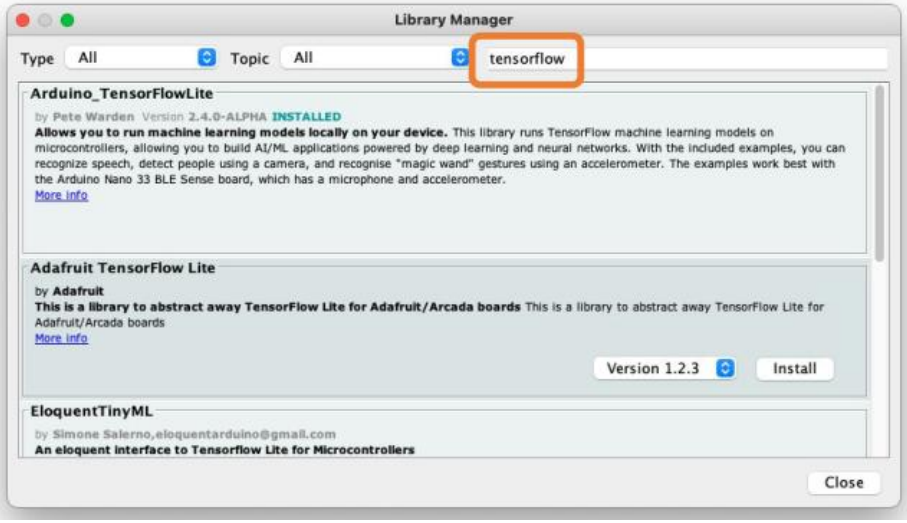


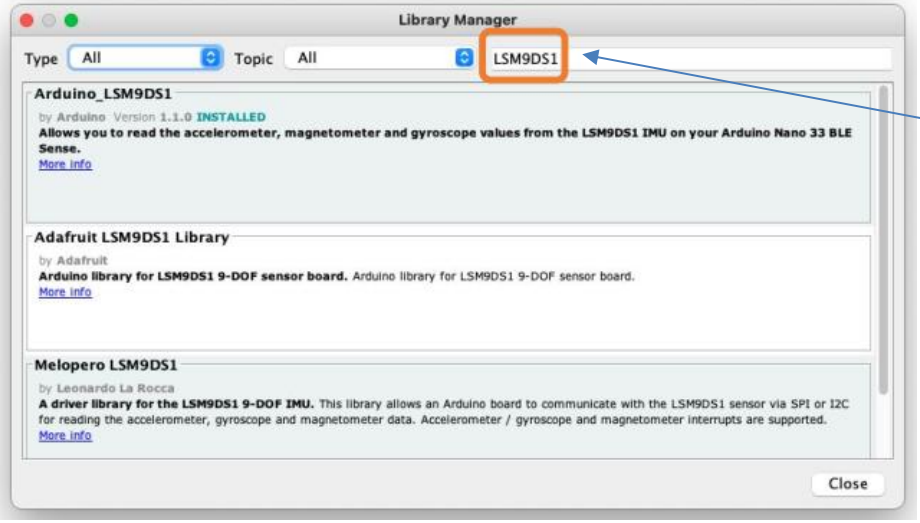
Arduino nano BLE 33 sense

Installing the Main Libraries





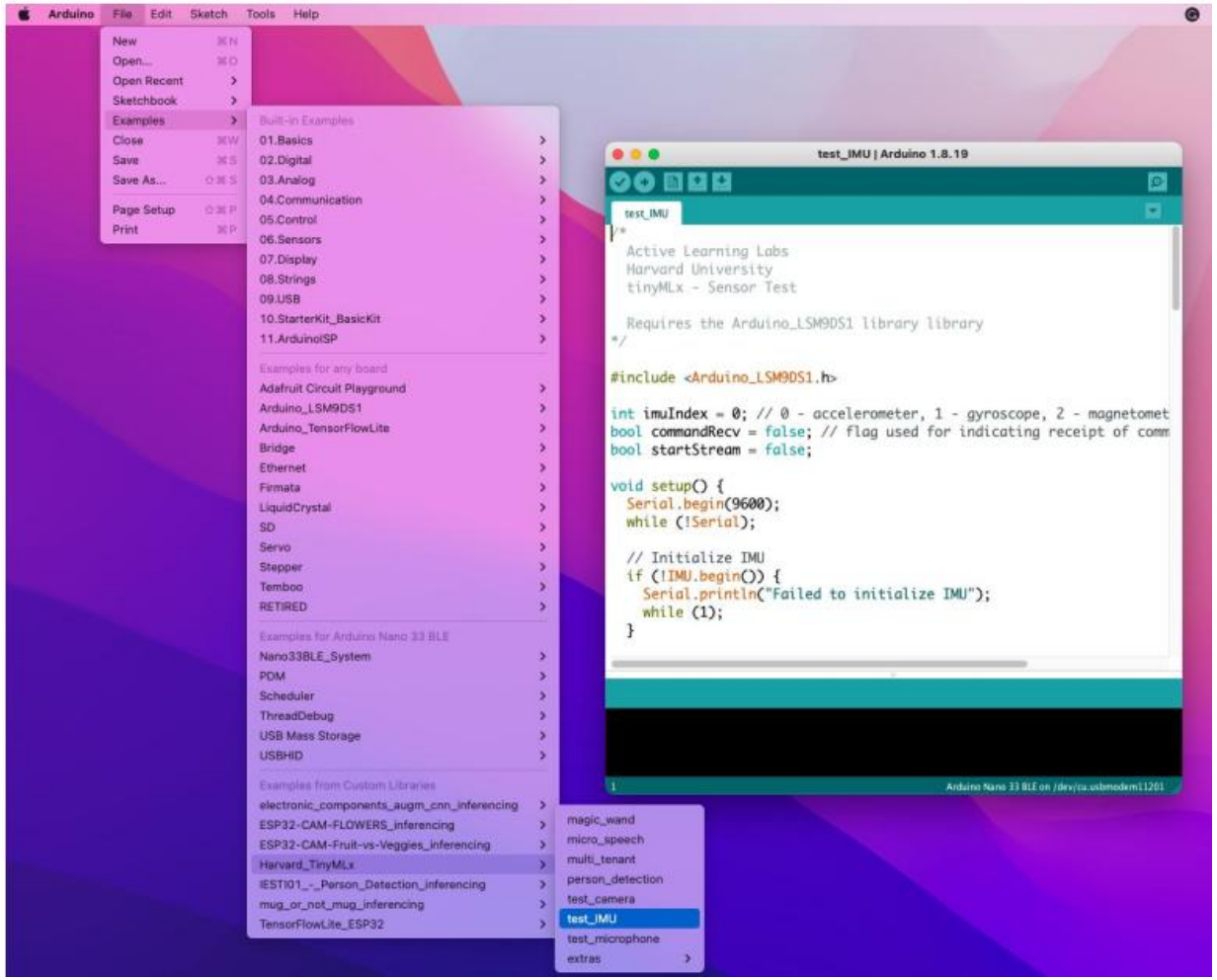


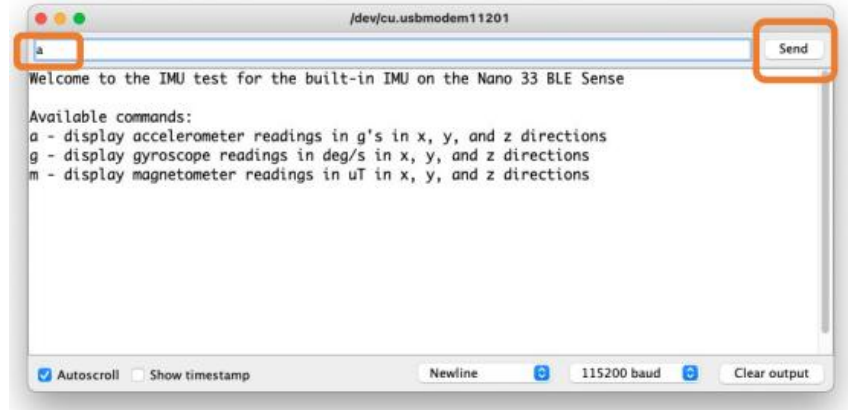


Lib para magnetômetro, acelerômetro e giroscópio.

Arduino nano BLE 33 sense

Exemplo: IMU



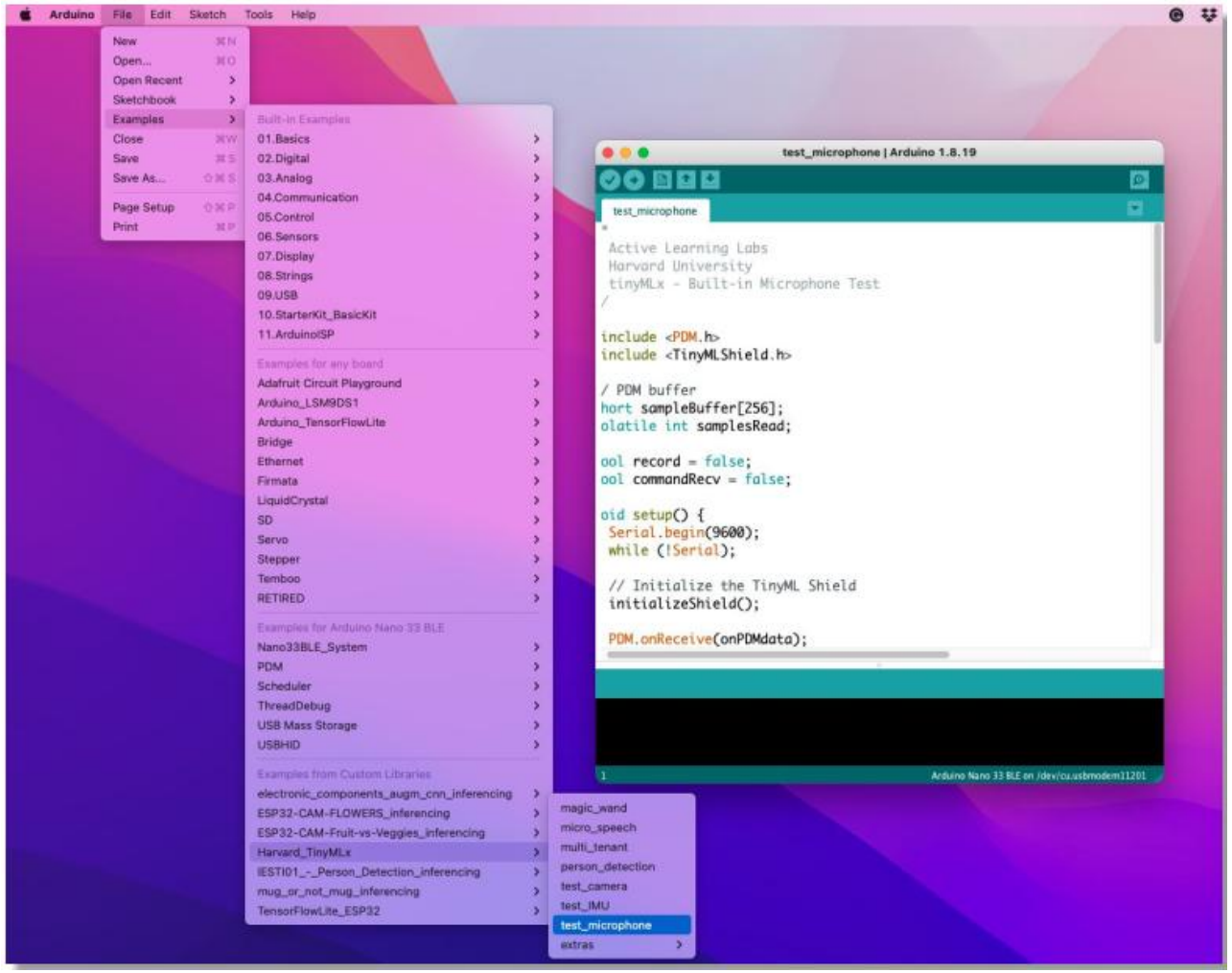


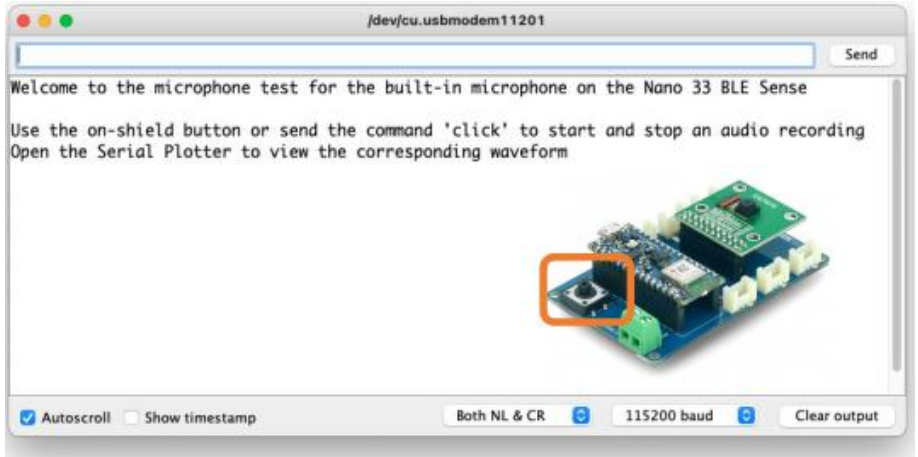


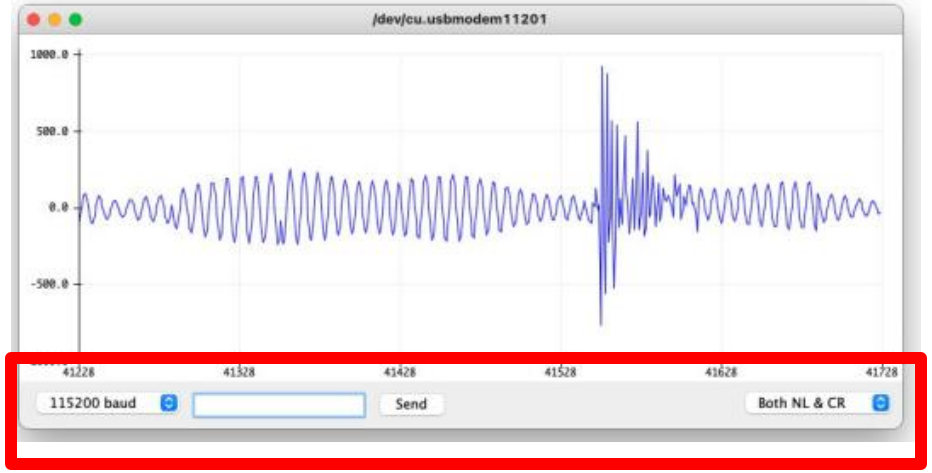
Notes: Close the Serial Monitor before open the Plotter
Repeat test for 'g' and 'm'

Arduino nano BLE 33 sense

Exemplo: Microfone



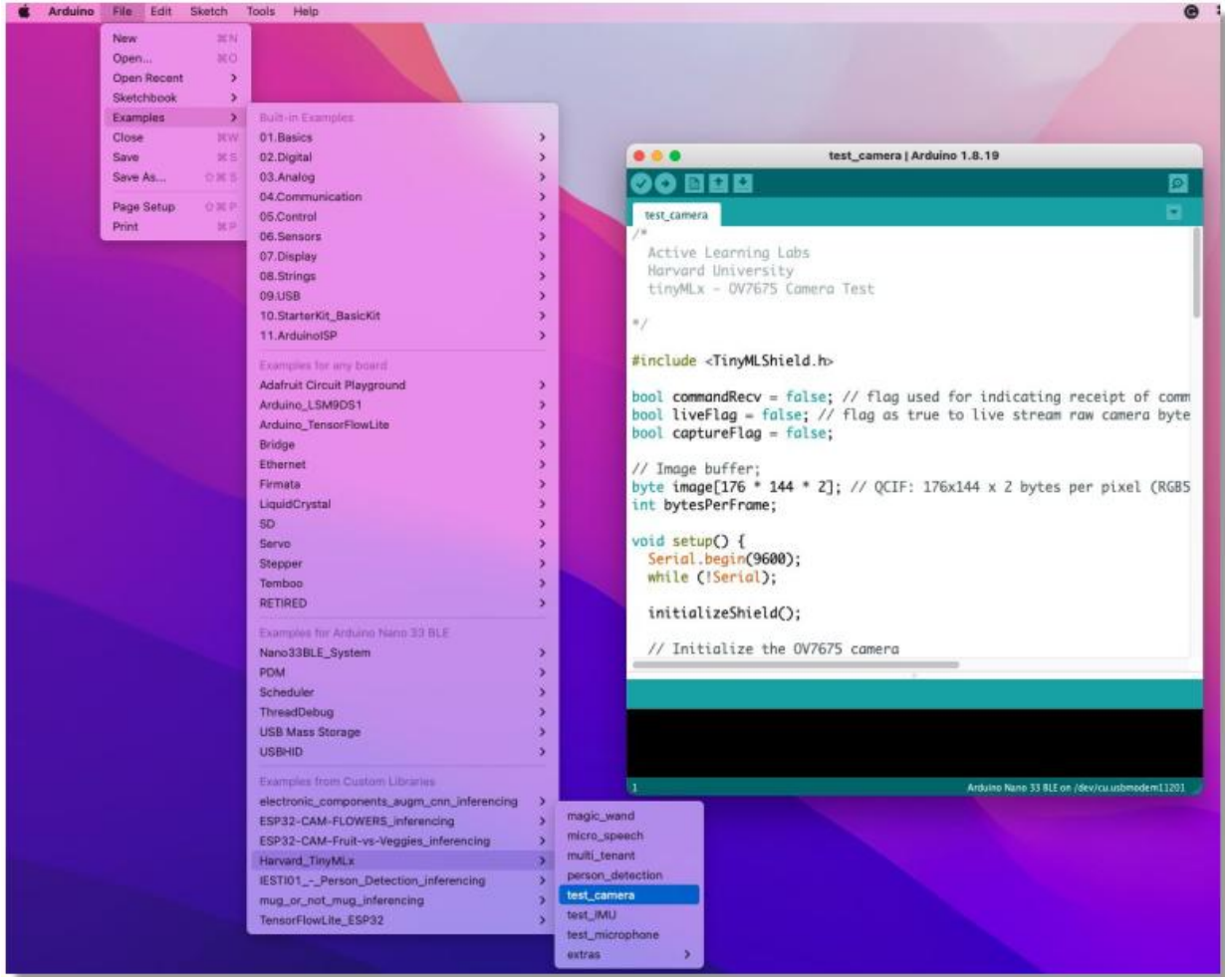




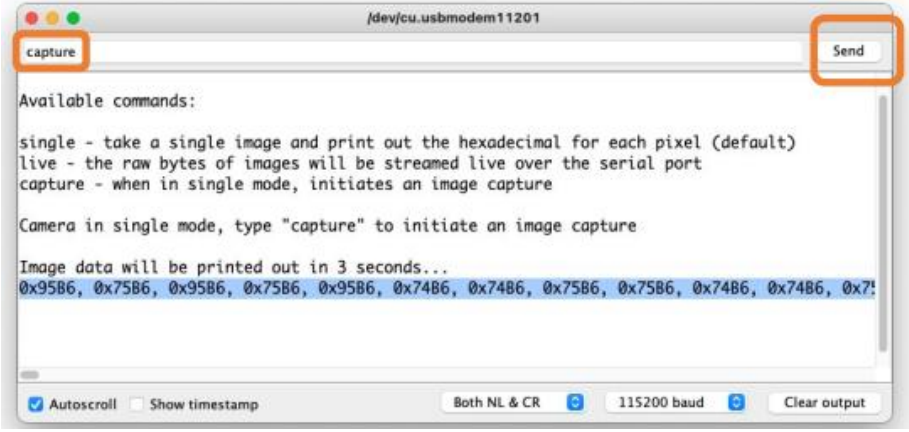
Note: Close the Serial Monitor before open the Plotter

Arduino nano BLE 33 sense

Exemplo: Câmera





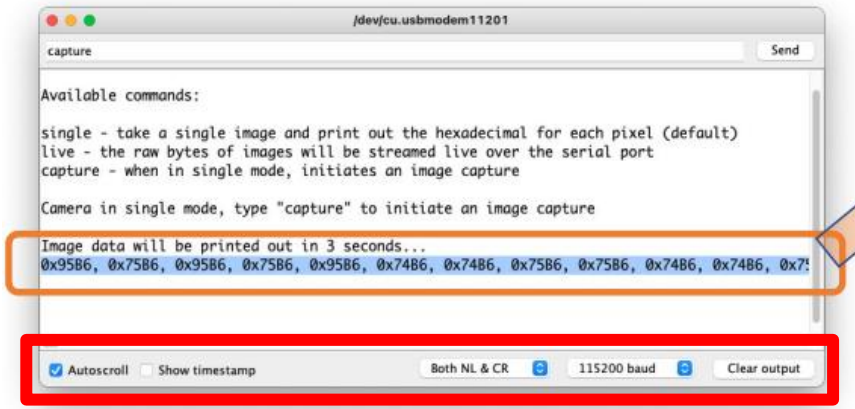


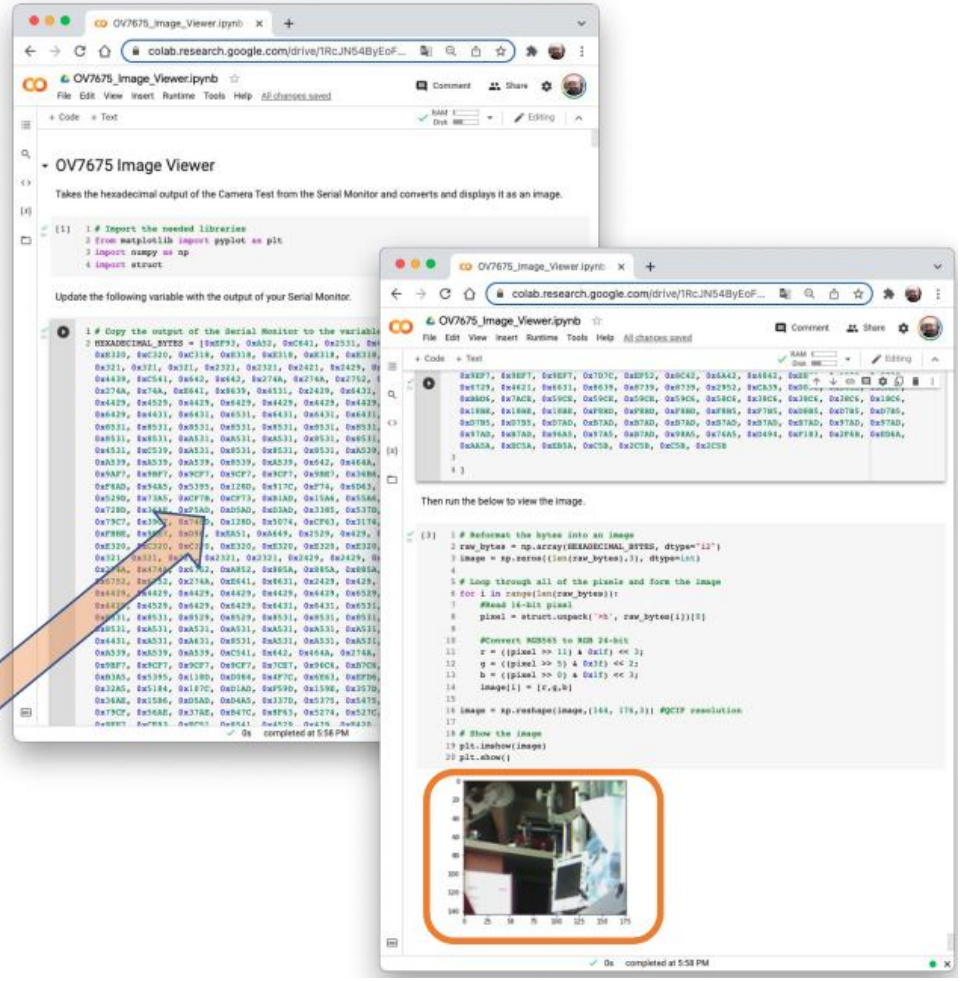
Note: You can Press Button insteady send ‘capture’

Arduino nano BLE 33 sense

Exemplo: Câmera







https://colab.research.google.com/github/zz4fap/tp557-iot-ml/blob/main/examples/4_2_12_OV7675ImageViewer.ipynb

Edge Impulse

- Tarefa para casa
  - Execute um dos projetos criados na última aula no Arduino nano BLE 33 sense.